

AZURE AI / ML

PRACTITIONER HANDBOOK

Seventh Edition — The Definitive Azure AI Reference

For Python developers with no cloud experience → Azure AI/ML Practitioner

Subscription · Identity · Networking · Key Vault · OpenAI · AI Search · AI Foundry

Container Registry · Container Apps · App Service · Function App · API Management · Log Analytics

88 Visuals	9 Auth Types	5 AI Use Cases	Portal Walkthroughs	Complete Examples
------------	--------------	----------------	---------------------	-------------------

Table of Contents

PART 1: GETTING STARTED

Chapter 1: Azure Portal — Navigation, Cloud Shell & First Login 4

- 1.1 What the Azure Portal is and how to navigate it 4
- 1.2 Azure Portal vs Azure AI Foundry — which to use when 4
- 1.3 Azure Cloud Shell — browser-based terminal 5
- 1.4 Day-1 checklist 5

Chapter 2: Subscriptions, Tenants & Resource Groups 6

- 2.1 Azure hierarchy: Tenant → Subscription → RG → Resource 6
- 2.2 Create Resource Group — portal walkthrough 7
- 2.3 Register Resource Providers 7

PART 2: AUTHENTICATION — COMPLETE GUIDE

Chapter 3: Authentication — All 9 Types Explained 9

- 3.1 Key terminology: OAuth 2.0, OIDC, JWT, Bearer Token, PKCE 9
- 3.2 Authentication types overview and decision guide 10
- 3.3 Type 1: System-assigned Managed Identity (most common) 11
- 3.4 Type 2: User-assigned Managed Identity 13
- 3.5 Type 3: App Registration & Service Principal + OIDC 14
- 3.6 Type 4: OAuth 2.0 Auth Code + PKCE (user-interactive) 16
- 3.7 RBAC roles reference — minimum roles for each service 17

PART 3: NETWORKING

Chapter 4: VNet, Subnets, NSG & Private Endpoints 19

- 4.1 VNet architecture — 8 subnets for the complete AI stack 19
- 4.2 Create VNet + subnets — portal walkthrough 20
- 4.3 NSG rules — what they are and how to configure 21
- 4.4 Private Endpoints — concept and complete portal walkthrough 22

PART 4: AZURE SERVICES — SETUP, CONFIGURE & USE

Chapter 5: Azure Key Vault 25

Chapter 6: Azure Container Registry (ACR) 27

Chapter 7: Container Apps Environment & Container Apps 28

Chapter 8: App Service & Azure Functions 31

Chapter 9: Storage Account — Blob & ADLS Gen2 33

Chapter 10: API Management 34

Chapter 11: Azure AI Search 36

Chapter 12: Azure AI Foundry — Models, Playground & Quotas 38

Chapter 13: Log Analytics Workspace & Application Insights 40

Chapter 14: Cost Management 42

Chapter 15: Troubleshooting — Common First-Day Errors 43

PART 5: AI USE CASES

Chapter 16: RAG Chatbot — Enterprise Knowledge Q&A	 45
Chapter 17: Document Intelligence Pipeline	 47
Chapter 18: AI-Powered Customer Support	 48
Chapter 19: Semantic Search	 49
Chapter 20: Batch Content Generation	 50

APPENDICES

Appendix A: CLI Quick Reference	 51
Appendix B: Python SDK Quick Reference	 52
Appendix C: Azure AI Service Relationships Map	 53

PART 1: GETTING STARTED

Chapter 1: Azure Portal — Navigation & First Login

If you have never used Azure before, start here. This chapter covers how to navigate the Azure Portal, use Cloud Shell without installing anything locally, and understand which portal URL to use for which task.

1.1 The Azure Portal

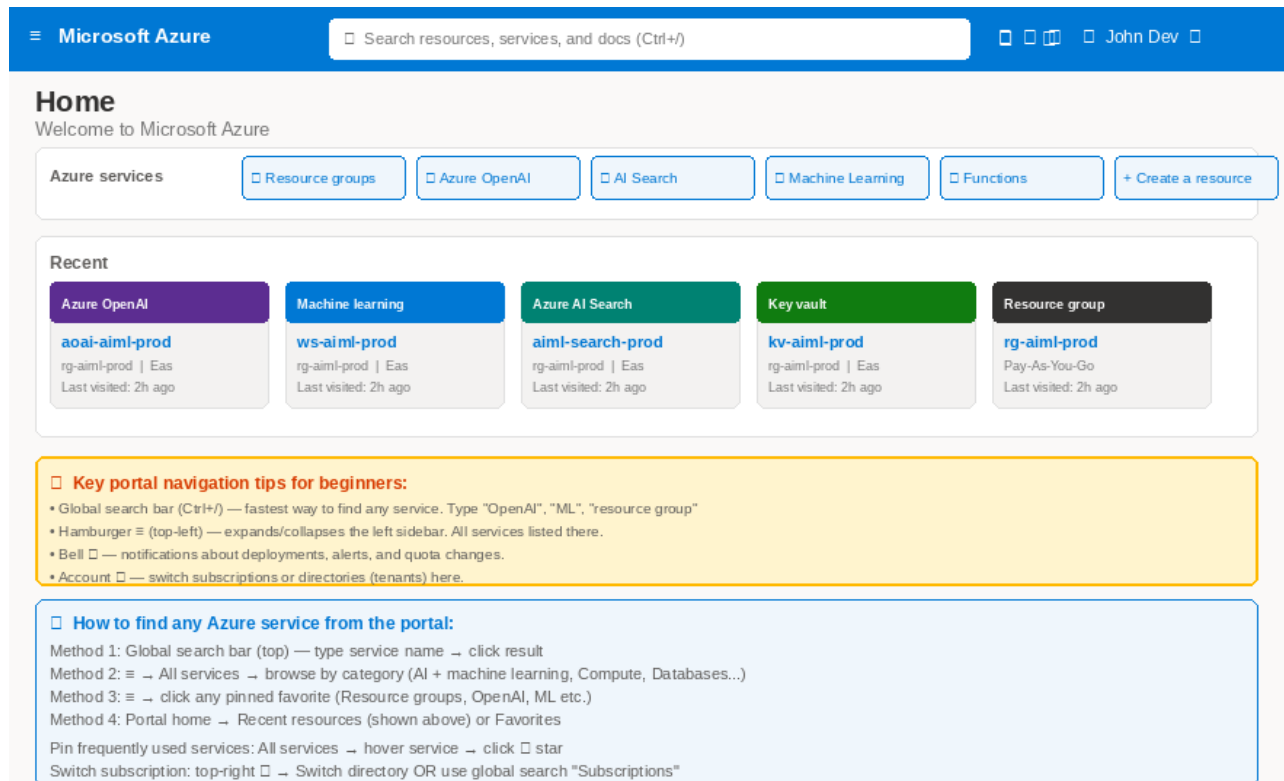


Fig 1.1 — Azure Portal home: search bar, recent resources, navigation and tips

The Azure Portal (portal.azure.com) is your browser-based control plane. Everything in Azure can be created, configured, and monitored here.

Element	Where to find it	What it does
Search bar (Ctrl+/)	Top centre of page	Fastest navigation — type any service name: "openai", "key vault", "resource group"
+ Create a resource	Home page or search results	Wizard to provision any new Azure service
≡ Hamburger menu	Top left corner	Expands left sidebar — All services organised by category
Bell 🔔 Notifications	Top right	Deployment complete, alerts firing, quota changes
Account 👤	Top right	Switch subscriptions, switch tenants (directories), sign out
Cloud Shell >_	Top right icon bar	Browser bash terminal — pre-authenticated, nothing to install
Favourites (sidebar)	Left navigation bar	Pin most-used services: star them in "All services"
Recent (home page)	Home page cards	Last resources visited — fastest way to return to work

1.2 Azure Portal vs AI Foundry — Which to Use When

Two Portals: Azure Portal vs Azure AI Foundry — What to Use for What

portal.azure.com Azure Portal — Infrastructure & IAM	ai.azure.com Azure AI Foundry — Model & Deployment Management
☐ Resource provisioning Create OpenAI, Storage, VNet, AKS, etc.	☐ Model deployments Deploy GPT-4o, mini, embeddings, DALL-E, Whisper
☐ Identity & access (IAM) Role assignments, managed identity, RBAC	☐ Chat playground Test prompts interactively before writing code
☐ Networking VNet, subnets, NSG, private endpoints, DNS	☐ Quotas management View and request TPM/RPM quota increases
☐ Cost management Billing, cost analysis, budgets, alerts	☐ Content filters Configure harmful content filtering profiles
☐ Azure Monitor Metrics, logs, alerts, dashboards, workbooks	☐ Usage & metrics Token usage, latency, error rates per deployment
☐ Resource configuration Settings, diagnostics, backups, scaling	☐ Evaluation Run automated evals on RAG quality metrics
☐ Service health Incident history, planned maintenance	☐ AI Agents Create, test, manage agentic AI systems
☐ Key Vault Secrets, certificates, keys management	☐ Prompt flow Visual RAG and agent pipeline builder

Fig 1.2 — Side-by-side comparison of Azure Portal vs Azure AI Foundry

If you want to...	Use this	URL
Create any resource (OpenAI, VNet, Storage, etc.)	Azure Portal	portal.azure.com → + Create a resource
Assign RBAC roles and manage identities	Azure Portal	Resource → Access control (IAM) → + Add role assignment
Configure networking, NSG, private endpoints	Azure Portal	Resource → Networking tab
View costs and set budget alerts	Azure Portal	Cost Management → Cost analysis
Deploy GPT-4o or text-embedding-3-large model	Azure AI Foundry	ai.azure.com → Deployments → + Deploy model
Test prompts before writing code	Azure AI Foundry	ai.azure.com → Chat playground
Request more TPM quota	Azure AI Foundry	ai.azure.com → Quotas → Request increase
Configure content safety filters	Azure AI Foundry	ai.azure.com → Content safety

1.3 Azure Cloud Shell

Microsoft Azure

Cloud Shell

Azure Cloud Shell — browser-based terminal (no local install needed)
 Access: portal.azure.com → click the Cloud Shell icon (>) in the top navigation bar

```

Bash | Upload/Download | Open editor | Restart | Settings

PS /home/john> az login --use-device-code
To sign in, use a web browser to open https://microsoft.com/devicelogin
and enter the code AB3CD4EF to authenticate.
PS /home/john> az account show --query "{name:name, id:id}" -o table
Name                SubscriptionId
Pay-As-You-Go        xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
PS /home/john> az group create --name rg-ai-ml-prod --location eastus2
{
  "id": "/subscriptions/xxx/resourceGroups/rg-ai-ml-prod",
  "location": "eastus2",
  "name": "rg-ai-ml-prod",
  "properties": {"provisioningState": "Succeeded"}
}
  
```

Cloud Shell gives you a pre-authenticated bash terminal in your browser.

- No local installation needed — az CLI, Python, git, kubectl all pre-installed
- Already logged in to your subscription — no az login needed
- 5 GB persistent storage in Azure Files — your scripts persist across sessions
- Access: Portal top nav ">" icon OR go directly to shell.azure.com

Fig 1.3 — Azure Cloud Shell: pre-authenticated browser bash terminal

Cloud Shell gives you a bash terminal in your browser. No local installation needed. It is already authenticated to your Azure subscription.

1. Open: portal.azure.com → click >_ icon in the top navigation bar → select Bash.
2. First time: Azure creates a storage account (5 GB) to persist your files between sessions.
3. Pre-installed: az CLI, Python 3, pip, git, kubectl, helm, terraform, jq, vim, curl.
4. Files: save scripts to ~/clouddrive — they persist across sessions.
5. Tip: go directly to shell.azure.com for faster access.

```

# Verify your setup — run these first in Cloud Shell
az account show --query "{name:name, sub:id, tenant:tenantId}" -o table

# Set defaults to save typing --resource-group and --location every time
az configure --defaults group=rg-ai-ml-prod location=eastus2

# Install Azure AI Python SDKs
pip install -q azure-identity openai azure-search-documents \
    azure-keyvault-secrets azure-storage-blob azure-ai-documentintelligence

# Test DefaultAzureCredential (works because Cloud Shell is pre-authenticated)
python3 -c "
from azure.identity import DefaultAzureCredential
tok = DefaultAzureCredential().get_token('https://management.azure.com/.default')
print('Auth OK — token length:', len(tok.token))
"
  
```

1.4 Day-1 Checklist

Day 1 Setup Checklist — In This Exact Order

Phase 1: Access & Tools (30 min) - Get access to Azure subscription from your admin (Subscription ID + Tenant ID) - Log in to portal.azure.com → verify you can see your subscription - Install Azure CLI locally OR open Cloud Shell (shell.azure.com — nothing to install) - Run: <code>az login && az account show</code> → verify correct subscription - Register all resource providers (see Chapter 1)	Phase 2: Foundation (2 hours) - Create resource group: <code>rg-ai-ml-dev</code> (for learning) and <code>rg-ai-ml-prod</code> (for production) - Create Virtual Network with all 8 subnets (see Chapter 3 — use East US 2) - Create Key Vault — enable RBAC permission model, disable public access - Create Storage Account — enable hierarchical namespace (ADLS Gen2) - Create Azure Container Registry (Premium for private endpoints)
Phase 3: AI Services (1 hour) - Create Azure OpenAI resource — region East US 2, public access: Disabled - In AI Foundry (ai.azure.com): deploy gpt-4o and text-embedding-3-large models - Create Azure AI Search — Standard S1 tier, enable semantic ranking - Test OpenAI: AI Foundry → Chat playground → send a test message - Test AI Search: portal → Search explorer → run a test query	Phase 4: Private Networking (1 hour) - Create private endpoints for each service → <code>snet-private-endpoints</code> subnet - Create private DNS zones and link to VNet (one zone per service type) - Disable public network access on each service - Test private endpoint: from Cloud Shell in VNet, <code>nslookup</code> up your OpenAI FQDN - Verify <code>nslookup</code> returns 10.0.4.x (private IP), not a public IP

Fig 1.4 — Four-phase Day-1 setup checklist — follow in order

Chapter 2: Subscriptions, Tenants & Resource Groups

2.1 The Azure Hierarchy

Azure Hierarchy: Tenant → Management Group → Subscription → Resource Group → Resource

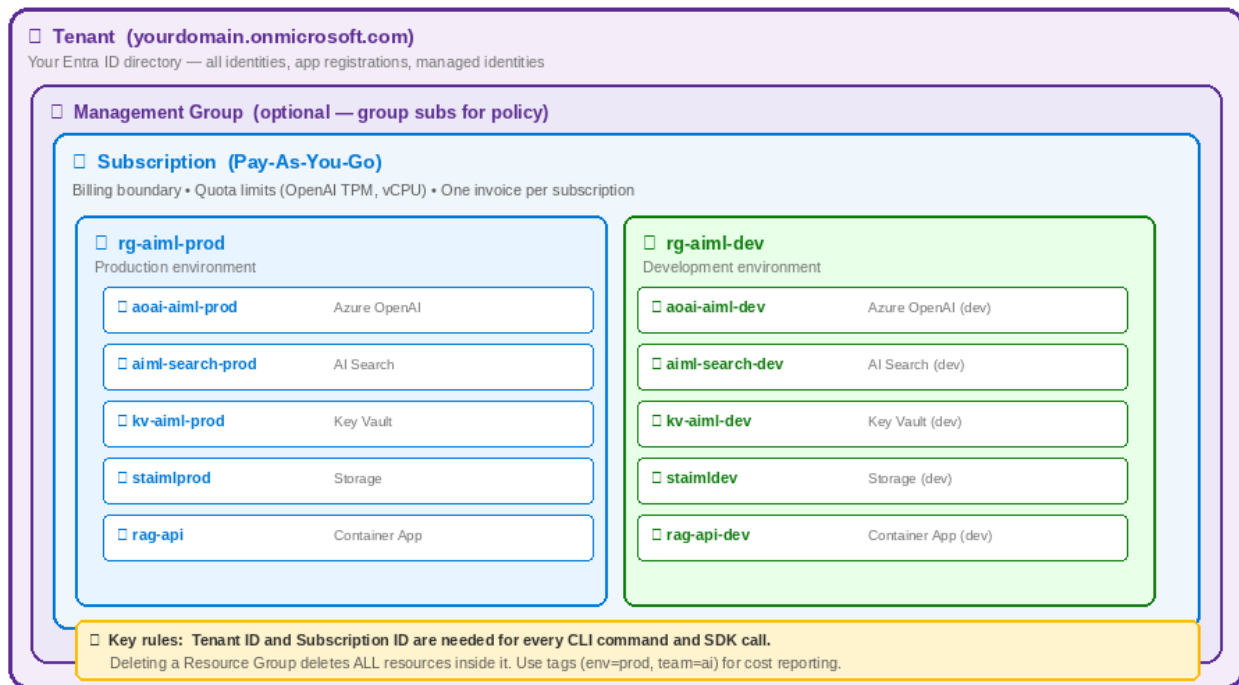


Fig 2.1 — Azure hierarchy: Tenant → Subscription → Resource Group → Resource

Key Terminology

Tenant (Entra ID)	Your company's Entra ID (formerly Azure AD) directory. All users, service principals, managed identities live here. One tenant per organisation. Get your Tenant ID: <code>az account show --query tenantId</code>	<i>Get it: <code>az account show</code></i>
Subscription	A billing account that groups your resources. One invoice per subscription. Resource limits (OpenAI TPM quota, vCPU count) are scoped to subscription + region. Get your Subscription ID: <code>az account show --query id</code>	<i>Get it: <code>az account show</code></i>
Resource Group (RG)	A logical container for resources that share the same lifecycle and environment. DELETING AN RG DELETES ALL RESOURCES INSIDE IT. Convention: <code>rg-{project}-{env}</code>	<i>Create: <code>az group create</code></i>
Resource	A single Azure service instance — one OpenAI account, one Storage account, one Container App. Always lives inside an RG. Has an ARM Resource ID.	<i>List: <code>az resource list</code></i>
ARM Resource ID	The unique path to any Azure resource: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/{namespace}/{type}/{name}</code> . Used in RBAC scope assignments and policy.	<i>Get: <code>resource → JSON view</code></i>
Region	A geographic area (e.g. East US 2) hosting Azure data centres. Choose East US 2 for widest Azure OpenAI model availability.	<i>Recommended: <code>eastus2</code></i>
Resource Provider	A namespace that enables a class of resources. Must be registered before you can create resources of that type. E.g. <code>Microsoft.CognitiveServices</code> for OpenAI.	<i>Register: <code>az provider register</code></i>

2.2 Create Resource Group — Portal Walkthrough

1	Navigate Portal → type "Resource groups" in search bar → click Resource groups → + Create
2	Fill Basics tab Subscription: select yours Name: rg-aiml-prod Region: (US) East US 2
3	Add tags Tags tab: env = prod team = ai-ml cost-center = ai001 → Review + create → Create

Fig 2.2 — Resource Group creation form with naming convention and tag guidance

2.3 Register Resource Providers

1	Navigate Portal → Subscriptions → click your subscription → Settings → Resource providers
2	Register Find each provider → click its name → click "Register" → wait ~1 minute → status shows "Registered"

Microsoft Azure

Home > Subscriptions > Pay-As-You-Go

Subscriptions

Overview

Access control (IAM)

Resource providers

Policies

Cost analysis

+ Add provider

Refresh

Filter by name...

Namespace	Status	Registration policy	Last modified
Microsoft.CognitiveServices	Registered	RegistrationRequired	2024-11-15
Microsoft.MachineLearningServices	Registered	RegistrationRequired	2024-11-10
Microsoft.Search	Registered	RegistrationRequired	2024-11-12
Microsoft.ContainerRegistry	Registered	RegistrationRequired	2024-11-01
Microsoft.App	Registered	RegistrationRequired	2024-11-05
Microsoft.ContainerService	Registered	RegistrationRequired	2024-10-28
Microsoft.KeyVault	Registered	RegistrationRequired	2024-10-20
Microsoft.Storage	Registered	RegistrationRequired	2024-10-15
Microsoft.DocumentDB	NotRegistered	RegistrationRequired	—
Microsoft.EventHub	NotRegistered	RegistrationRequired	—

Required step: Register all providers listed above before creating AI resources.

Unregistered providers will cause "SubscriptionNotRegistered" errors when provisioning.

Fig 2.3 — Resource Providers page showing registered and unregistered providers

Provider	Register for	Status needed
Microsoft.CognitiveServices	Azure OpenAI, AI Vision, Speech, Document Intelligence	Registered
Microsoft.MachineLearningServices	Azure ML Workspace, compute clusters	Registered
Microsoft.Search	Azure AI Search	Registered
Microsoft.App	Container Apps and environments	Registered
Microsoft.ContainerRegistry	ACR — private image registry	Registered
Microsoft.KeyVault	Azure Key Vault	Registered
Microsoft.DocumentDB	Cosmos DB	Registered if using Cosmos
Microsoft.ServiceBus	Service Bus messaging	Registered if using SB

```

# Register all at once in Cloud Shell
for P in Microsoft.CognitiveServices Microsoft.MachineLearningServices \
    Microsoft.Search Microsoft.App Microsoft.ContainerRegistry \
    Microsoft.KeyVault Microsoft.DocumentDB Microsoft.ServiceBus; do
    az provider register --namespace $P && echo "Registered: $P"
done

# Verify — all should show "Registered"
az provider list --query "[?registrationState=='Registered'].namespace" -o tsv | sort | grep -E
"Cognitive|Machine|Search|App|Container|Vault|Document|Service"

```

PART 2: AUTHENTICATION — THE COMPLETE GUIDE

Chapter 3: Authentication — All 9 Types Explained

Authentication is the most critical concept to master in Azure. Getting it right means zero secrets in your code. Getting it wrong means security breaches or constant access failures. This chapter explains every authentication type in depth — what it is, how it works, and when to use it.

3.1 Key Terminology — OAuth 2.0, OIDC, JWT, Bearer Token

OIDC, OAuth 2.0 and JWT — Complete Terminology Explained

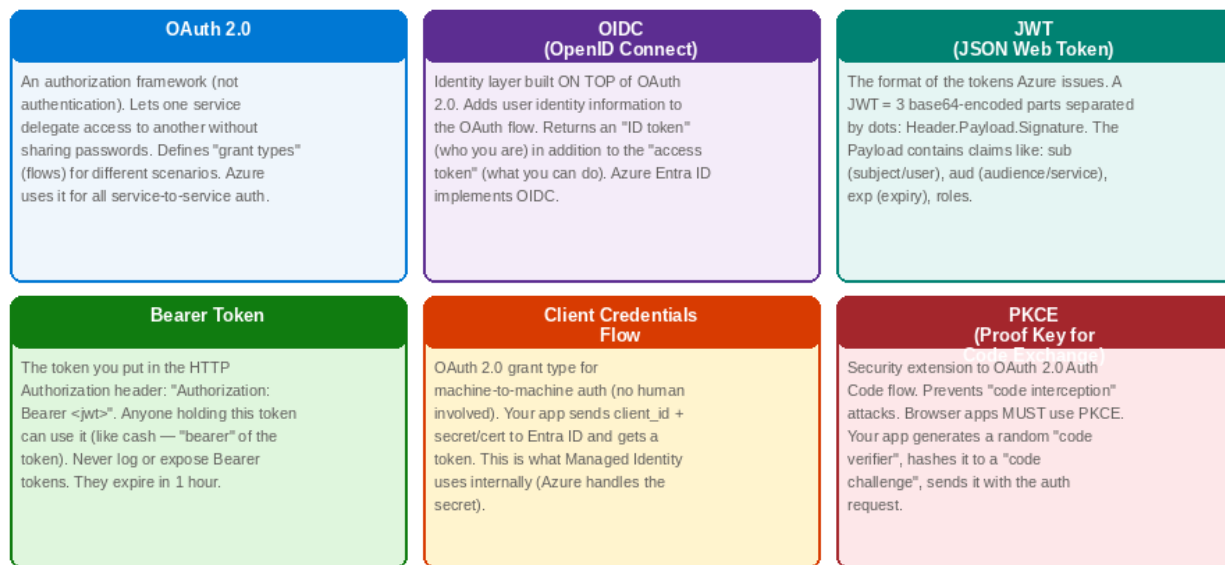


Fig 3.1 — OAuth 2.0, OIDC, JWT, Bearer Token, Client Credentials, PKCE — all explained

Understanding these terms helps you debug authentication failures and choose the right auth type:

OAuth 2.0	An authorization framework (not authentication). Defines how one service can delegate access to another without sharing passwords. Azure uses OAuth 2.0 for ALL service-to-service auth. Entra ID is the OAuth 2.0 authorization server.
OIDC (OpenID Connect)	Identity layer built ON TOP of OAuth 2.0. Adds user identity ("who are you") to OAuth. Returns an ID token alongside the access token. GitHub Actions uses OIDC to get tokens from Entra ID without needing a stored secret.
JWT (JSON Web Token)	The format of tokens issued by Entra ID. Three base64-encoded parts: Header (algorithm) + Payload (claims: who, what, when expires) + Signature (cryptographic proof). Your app never creates JWTs — Entra ID does.
Bearer Token	A JWT in the HTTP Authorization header: "Authorization: Bearer eyJ0eXAi...". Anyone holding it can use it. Tokens expire in 1 hour. The SDK auto-refreshes them. NEVER log or expose Bearer tokens.
Client Credentials Flow	OAuth 2.0 grant type for machine-to-machine auth with no human user involved. App sends client_id + secret/cert to Entra ID → gets access token. Managed Identity uses this flow internally — Azure manages the credential for you.
PKCE (Proof Key for Code)	Security extension for OAuth 2.0 Auth Code flow. Prevents "code interception" attacks. Required for all browser/mobile apps. App generates a random code_verifier, hashes it as code_challenge,

Exchange)	sends both as proof of identity.
Scope	What the token grants access to. Format: "https://cognitiveservices.azure.com/.default" means full access to Azure Cognitive Services. Scopes are listed in the app registration.
Federated Credential (OIDC federation)	A trust relationship between a platform (GitHub, AKS) and an App Registration. GitHub Actions can get an Entra ID token by presenting its own OIDC token — no client secret needed at all.

3.2 Authentication Types — Overview & Decision Guide

Azure Authentication — All Types: What They Are, When to Use, How They Work

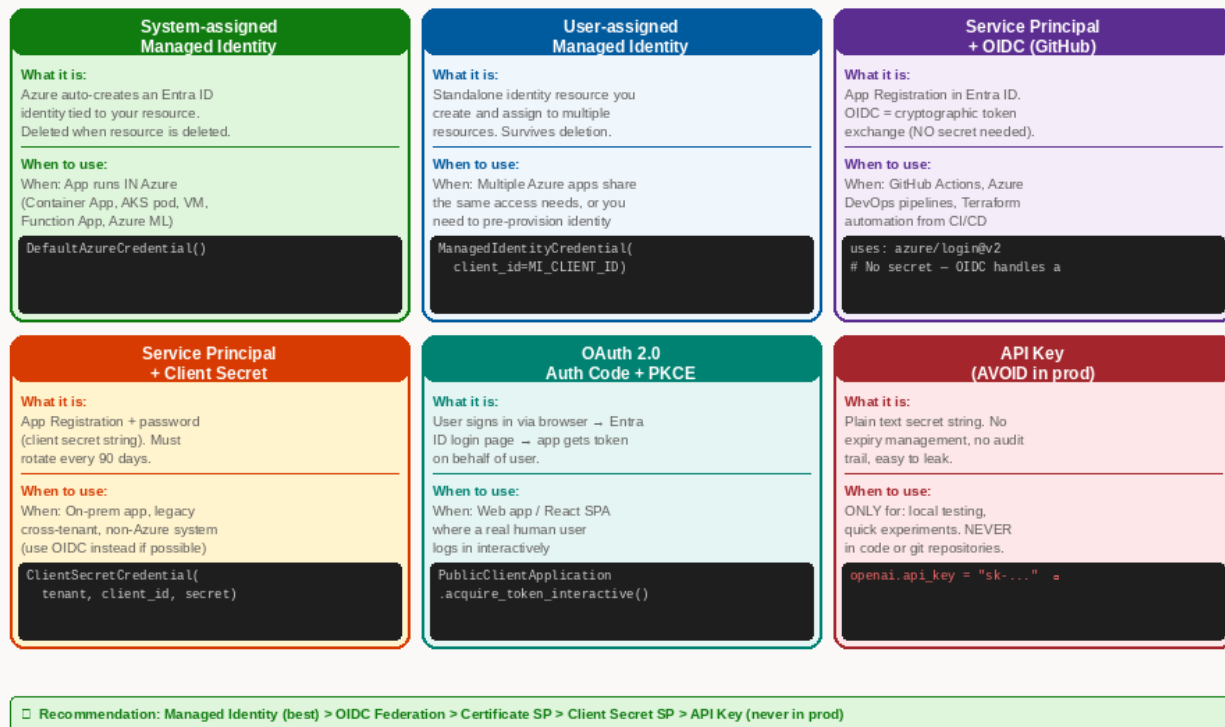


Fig 3.2 — All 6 authentication types: what they are, when to use, code example

Authentication Type Decision — Which Auth to Use for Each Scenario



Fig 3.3 — Decision flowchart: which auth type to choose for each scenario

Auth type	When to use	Secret management	Security rating
System-assigned MI	App runs IN Azure (ACA, AKS pod, VM, Functions)	None — Azure handles everything	★★★★★ Best
User-assigned MI	Multiple Azure apps share access, pre-provision identity	None — Azure handles everything	★★★★★ Best
OIDC Federation	GitHub Actions, Azure DevOps, AKS workload identity	None — cryptographic exchange	★★★★★ Best

Auth type	When to use	Secret management	Security rating
SP + Certificate	On-prem app calling Azure, cross-tenant access	Certificate (auto-rotate via KV)	★★★★ Good
SP + Client Secret	Legacy CI/CD, non-Azure (use OIDC instead)	String password (rotate every 90d)	★★★ OK
OAuth 2.0 + PKCE	User signs in via browser (SPA, web app)	User credential in Entra ID	★★★★ Good
Client Credentials	Background service calls another service (no user)	Client secret or certificate	★★★ OK
API Key	NEVER in production — local experiments only	You manage manually	★ Avoid

3.3 Type 1: System-assigned Managed Identity

The most common auth type for Azure workloads. Azure creates a service principal in Entra ID for your resource and rotates its credentials automatically. Your code uses `DefaultAzureCredential()` — no config needed.

How Azure Authentication Works — Token Flow (Managed Identity example)

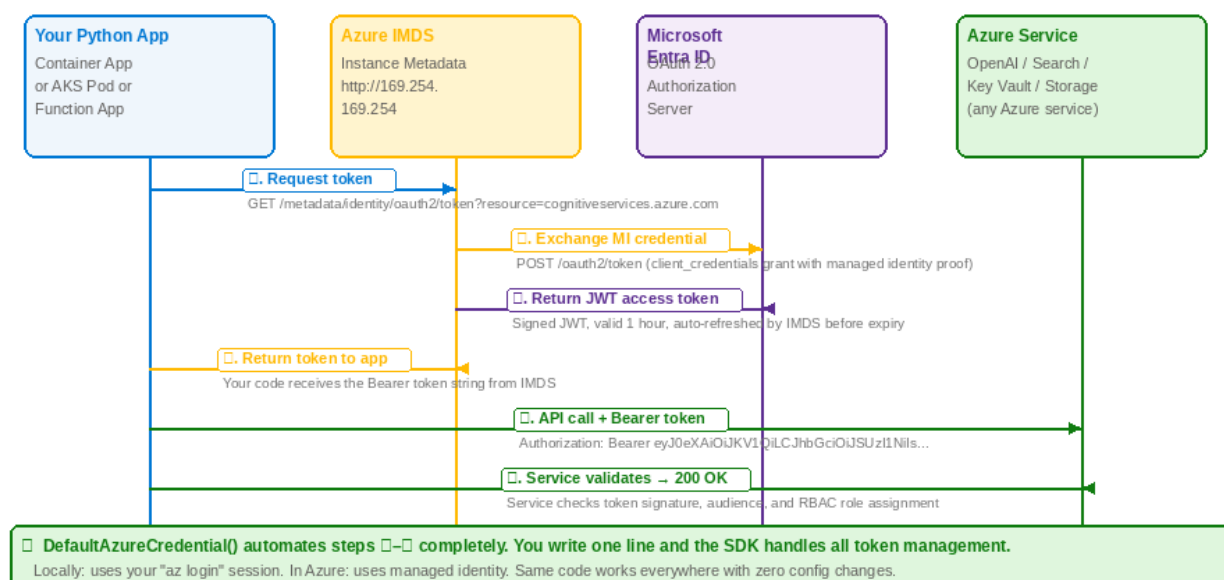


Fig 3.4 — Token flow: how `DefaultAzureCredential()` gets a token for your app automatically

How it works — step by step

- You enable system-assigned MI on your Container App / AKS pod / VM / Function.
- Azure creates a service principal in Entra ID and stores its credential in IMDS (Instance Metadata Service).
- When `DefaultAzureCredential()` is called, it contacts IMDS at `http://169.254.169.254` (internal Azure IP).
- IMDS exchanges the managed identity credential with Entra ID and returns a JWT access token.
- The SDK attaches the token to your API call: `Authorization: Bearer <jwt>`.
- The target service (OpenAI, Search, etc.) validates the token and checks your RBAC role assignment.
- Token is cached and auto-refreshed before expiry — you never manage tokens yourself.

1 Enable MI on your resource

Portal → Container Apps → your app → Identity (left sidebar) → System assigned tab → toggle Status to On → Save

The screenshot shows the Azure Portal interface for a Container App's Identity tab. The breadcrumb trail is 'Home > Container Apps > rag-api > Identity'. The 'Identity' tab is selected, with sub-tabs for Overview, Deployments, Ingress, Scale and replicas, Identity, and Monitoring. Under the 'Identity' sub-tab, there are two buttons: 'System assigned' (active) and 'User assigned'. The 'System assigned managed identity' section shows that the status is 'On' (indicated by a blue toggle switch). Below this, the 'Object (principal) ID' is listed as '7f3a8c2b-1e4d-4f9a-8b3c-2d1e5f6a7b8c' and the 'Client ID' is 'a1b2c3d4-e5f6-7890-abcd-ef1234567890'. A 'Save' button is present. The 'Azure role assignments' section includes a button labeled 'Azure role assignments →' and a CLI command: 'Or use CLI: az role assignment create --assignee <principal-id> --role "<role-name>" --scope <resource-id>'. At the bottom, a green box lists 'Current role assignments for this identity:' with two entries: 'Cognitive Services OpenAI User → aoai-ai-ml-prod (allows calling GPT-4o, embeddings)' and 'Search Index Data Reader → ai-ml-search-prod (allows querying indexes)'.

Fig 3.5 — Container App Identity tab: enable system-assigned MI, view Principal ID, click Azure role assignments

2 Copy the Principal ID

After saving: copy the Object (principal) ID shown — you will use this for role assignments

3 Assign RBAC role

Click "Azure role assignments" button → Add role assignment → fill in Role, Members, Scope

Fig 3.6 — Add role assignment: select Managed identity, choose your app, verify scope is specific resource

Role assignment — click-by-click

13. Role tab: search for "Cognitive Services OpenAI User" → select → Next.
14. Members tab: "Assign access to" → choose "Managed identity" (never User or SP for Azure apps).
15. Members tab: Managed identity type → "Container Apps" → click "+ Select members" → find your app.
16. Review + assign tab: verify Scope shows the SPECIFIC RESOURCE (not subscription or RG) → click Review + assign.
17. Wait 2-5 minutes for RBAC to propagate before testing.

```
# Enable MI and assign all roles for a RAG API — run in Cloud Shell
RG="rg-aiml-prod"; APP="rag-api"

# Enable system-assigned MI on Container App
az containerapp identity assign -n $APP -g $RG --system-assigned

# Get Principal ID
MI=$(az containerapp show -n $APP -g $RG --query identity.principalId -o tsv)
echo "Principal ID: $MI"

# Get resource IDs for each service
AOAI=$(az cognitiveservices account show -n aoai-aiml-prod -g $RG --query id -o tsv)
SRCH=$(az search service show -n aiml-search-prod -g $RG --query id -o tsv)
KV=$(az keyvault show -n kv-aiml-prod -g $RG --query id -o tsv)
SA=$(az storage account show -n staimlprod -g $RG --query id -o tsv)

# Assign minimum required roles — least privilege principle
```

```

az role assignment create --role "Cognitive Services OpenAI User" --assignee $MI --scope $AOAI
az role assignment create --role "Search Index Data Reader" --assignee $MI --scope $SRCH
az role assignment create --role "Key Vault Secrets User" --assignee $MI --scope $KV
az role assignment create --role "Storage Blob Data Reader" --assignee $MI --scope $SA

# Verify assignments
az role assignment list --assignee $MI --all -o table

# Python code — DefaultAzureCredential detects MI automatically when running in Azure
from azure.identity import DefaultAzureCredential
from azure.identity import get_bearer_token_provider
from openai import AzureOpenAI
import os

cred = DefaultAzureCredential() # no config needed — auto-detects MI

client = AzureOpenAI(
    azure_endpoint=os.environ["OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        cred, "https://cognitiveservices.azure.com/.default"),
    api_version="2024-08-01-preview"
)

```

3.4 Type 2: User-assigned Managed Identity

Managed Identity — System-assigned vs User-assigned: Full Comparison

System-assigned Managed Identity	User-assigned Managed Identity
Tied to: One specific Azure resource (1:1 relationship) Lifecycle: Created with resource; deleted when resource deleted Scope: Only that one resource can use this identity Setup: Toggle "On" in resource → Identity tab Cost: Free — no additional charge Best for: Single resource needs specific access Example: Container App → reads from specific Key Vault Object ID: Auto-generated, visible in Identity tab Rotation: Automatic — Azure manages credential rotation	Tied to: Standalone resource (1:many relationship) Lifecycle: Independent — survives resource deletion Scope: Assign to multiple resources Setup: Create MI resource, then assign to resources Cost: Free — no additional charge Best for: Shared access across multiple resources Example: 3 Container Apps all read same Storage account Client ID: Fixed GUID you can reference explicitly Pre-provision: Can create before the app resource exists
How to enable (portal steps): 1. Go to your resource (Container App, VM, Function App) 2. Left sidebar → Identity → System assigned tab 3. Toggle Status to "On" → Save 4. Copy the Object (principal) ID shown 5. Assign RBAC roles to that principal ID 6. In code: DefaultAzureCredential() — done!	How to enable (portal steps): 1. Portal → search "User assigned managed identities" 2. + Create → name: mi-rag-api, RG, region → Review+create 3. Go to your resource → Identity → User assigned tab 4. + Add → select your MI → Add 5. Assign RBAC roles to the MI's principal ID 6. In code: ManagedIdentityCredential(client_id=MI_CLIENT_ID)

Fig 3.7 — System-assigned vs User-assigned MI: properties, lifecycle, and portal setup steps

```

# Create user-assigned MI
az identity create --name mi-rag-shared -g rg-aiml-prod --location eastus2

# Get BOTH IDs — you need different ones for different purposes

```



```

MI_CLIENT=$(az identity show -n mi-rag-shared -g rg-aiml-prod --query clientId -o tsv)
MI_OBJECT=$(az identity show -n mi-rag-shared -g rg-aiml-prod --query principalId -o tsv)
# principalId = for RBAC role assignments
# clientId = for SDK calls when you want to specify which MI

# Assign roles (use principalId/objectId for RBAC, not clientId)
az role assignment create --role "Cognitive Services OpenAI User" \
  --assignee $MI_OBJECT --scope $AOAI_ID

# Assign MI to your Container App
az containerapp identity assign -n rag-api -g rg-aiml-prod \
  --user-assigned $(az identity show -n mi-rag-shared -g rg-aiml-prod --query id -o tsv)

# Python: explicitly specify which MI when you have multiple
from azure.identity import ManagedIdentityCredential
cred = ManagedIdentityCredential(client_id=MI_CLIENT_ID)

```

3.5 Type 3: App Registration & Service Principal

App Registration & Service Principal — Concepts, Relationship & When to Use

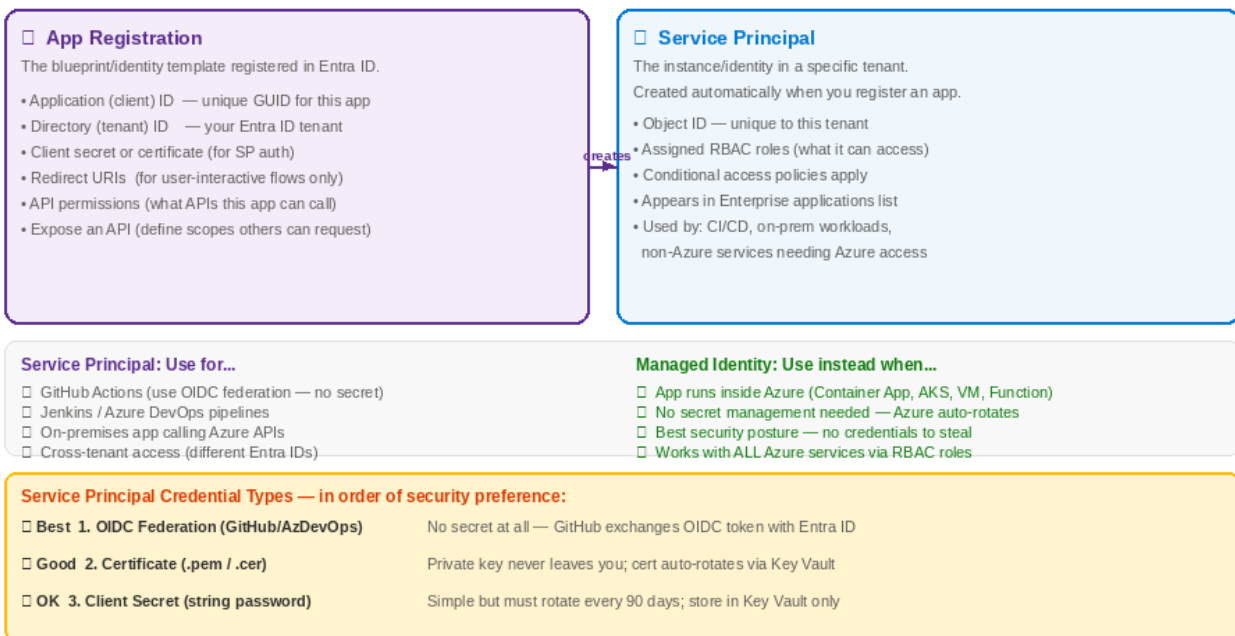


Fig 3.8 — App Registration vs Service Principal: what each is, credential types by security

1

Create App Registration

Entra ID → App registrations → + New registration → name, single tenant → Register

Microsoft Azure

Microsoft Entra ID > App registrations > New registration

Register an application

Microsoft Entra ID | yourtenant.onmicrosoft.com

Overview Authentication Certificates & secrets API permissions Expose an API

Name

Name *

rag-api-client

Display name shown to users when they consent to the app requesting permissions

Supported account types

☒ **Accounts in this organizational directory only (Single tenant)**
Only users in your Entra ID tenant. Choose for: internal enterprise apps, APIs used only by your org.

☐ **Accounts in any organizational directory (Multitenant)**
Users from any Entra ID tenant. Choose for: SaaS apps, third-party integrations.

☐ **Accounts in any org directory and personal Microsoft accounts**
Also allows Xbox, Outlook.com accounts. Rarely used for enterprise AI APIs.

Redirect URI (optional — for user-interactive flows)

Platform Redirect URI

Web ☐ https://localhost:3000/auth/callback

For API-only apps (machine-to-machine): leave blank. Only needed for user sign-in flows.

Register **Cancel**

Fig 3.9 — App Registration form: name, account types, redirect URI (blank for CI/CD)

2 Copy IDs

Overview page: copy Application (client) ID and Directory (tenant) ID — save both

3 Create credential

Certificates & secrets → + New client secret → description, 12 months → Add → COPY VALUE NOW

Microsoft Azure

Entra ID > App registrations > rag-api-client > Certificates & secrets

Certificates & secrets

Overview

Authentication

Certificates & secrets

API permissions

Expose an API

Certificates | Client secrets | Federated credentials

Client secrets

A secret string that the application uses to prove its identity when requesting a token. Also referred to as application password. Treat as a password — store in Key Vault, never in code.

+ New client secret

Description	Expires	Value (shown once)	Secret ID
prod-secret-2024	2025-11-20	Value hidden after creation	a8f2c1d3-...
github-actions-2024	2025-03-15	Value hidden after creation	b9e3d4f5-...

IMPORTANT: The secret value is shown only once, immediately after creation. Copy it now.

Best practice: Store in Azure Key Vault immediately. Set a calendar reminder 30 days before expiry.

Alternative (recommended): Use a certificate instead of a secret — certificates are harder to steal and can auto-rotate.

For Azure-to-Azure auth: skip this entirely. Use Managed Identity — no secret needed.

Only create client secrets for: GitHub Actions (use OIDC federation instead!), external apps, on-prem services.

Fig 3.10 — Certificates & secrets page: one-time copy warning, Key Vault best practice

✓ For GitHub Actions: use Workload Identity Federation — eliminates client secrets entirely. App Registration → Certificates & secrets → Federated credentials → Add → GitHub Actions → enter your org/repo/branch. GitHub gets an Entra ID token by presenting its own OIDC token. No client secret ever created.

```
# Configure GitHub Actions OIDC federation (no client secret needed)

# 1. Create federated credential on App Registration
az ad app federated-credential create \
  --id <app-registration-object-id> \
  --parameters '{
    "name": "github-main-branch",
    "issuer": "https://token.actions.githubusercontent.com",
    "subject": "repo:your-org/your-repo:ref:refs/heads/main",
    "audiences": ["api://AzureADTokenExchange"]
  }'

# 2. Store IDs as GitHub secrets (no client secret!)
gh secret set AZURE_CLIENT_ID      --body "<Application (client) ID>"
gh secret set AZURE_TENANT_ID      --body "<Directory (tenant) ID>"
gh secret set AZURE_SUBSCRIPTION_ID --body "<Subscription ID>"

# 3. GitHub Actions workflow
permissions:
  id-token: write # REQUIRED for OIDC
  contents: read

steps:
```

```
- uses: azure/login@v2
  with:
    client-id: ${ secrets.AZURE_CLIENT_ID }
    tenant-id: ${ secrets.AZURE_TENANT_ID }
    subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
    # No client-secret field – OIDC handles it cryptographically
```

3.6 Type 4: OAuth 2.0 Auth Code + PKCE (User Login)

For web apps and SPAs where real users sign in with their Entra ID (work) account. The user authenticates via Entra ID's login page. Your app receives a token on behalf of that user.

```
# Backend FastAPI: validate the incoming JWT from a logged-in user
import httpx, os
from jose import jwt, JWTError
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

TENANT_ID = os.environ["AZURE_TENANT_ID"]
CLIENT_ID = os.environ["AZURE_CLIENT_ID"] # Your API's App Registration

async def get_jwks():
    """Fetch Entra ID public keys for JWT signature verification"""
    async with httpx.AsyncClient() as h:
        oidc = await h.get(f"https://login.microsoftonline.com/{TENANT_ID}/v2.0/.well-known/openid-configuration")
        jwks = await h.get(oidc.json()["jwks_uri"])
        return jwks.json()

async def validate_token(token: str = Depends(HTTPBearer())) -> dict:
    """Validate JWT and extract claims (user info)"""
    try:
        payload = jwt.decode(
            token.credentials,
            key=await get_jwks(),
            algorithms=["RS256"],
            audience=CLIENT_ID # MUST match the "aud" claim in the token
        )
        return {
            "user_id": payload["oid"], # Entra ID object ID of the user
            "email": payload.get("preferred_username"),
            "name": payload.get("name"),
            "roles": payload.get("roles", []),
        }
    except JWTError as e:
        raise HTTPException(status_code=401, detail=f"Invalid token: {e}")

@app.post("/api/rag/query")
async def rag_query(body: QueryRequest, user=Depends(validate_token)):
    # user["user_id"] is the Entra ID object ID of the logged-in person
    return await do_rag(body.question, tenant_id=user["user_id"])
```

3.7 RBAC Roles Reference

RBAC Role Assignment Reference — Minimum Required Roles for Each Service

Rule: Always assign the MINIMUM role needed. Never use Owner or Contributor for application identities.

Service	Role name	What it allows	Assign to
Azure OpenAI	Cognitive Services OpenAI User	Call completions, embeddings, assistants APIs	App identity (read-only usage)
Azure OpenAI	Cognitive Services OpenAI Contributor	Also deploy/delete model versions	CI/CD pipelines
AI Search	Search Index Data Reader	Query indexes (search calls)	App that does search
AI Search	Search Index Data Contributor	Upsert/delete documents in index	Ingestion pipeline identity
AI Search	Search Service Contributor	Create/delete indexes, skillsets	IaC pipeline (one-time setup)
Storage	Storage Blob Data Reader	Read blobs (download documents)	App that reads documents
Storage	Storage Blob Data Contributor	Read + write blobs	Ingestion pipeline, ML training
Key Vault	Key Vault Secrets User	Read secret values	All application identities
Key Vault	Key Vault Secrets Officer	Create/update/delete secrets	CI/CD pipelines, admins
Container Registry	AcrPull	Pull container images	AKS, Container Apps, Functions
Container Registry	AcrPush	Push/build images to registry	CI/CD build pipelines
Document Intelligence	Cognitive Services User	Analyse documents	Ingestion function/app
Service Bus	Azure Service Bus Data Sender	Send messages to queues/topics	Producer apps
Service Bus	Azure Service Bus Data Receiver	Receive/peek messages	Consumer apps/functions
Cosmos DB	Cosmos DB Built-in Data Reader	Read documents	Read-only app queries
Cosmos DB	Cosmos DB Built-in Data Contributor	Read + write documents	App full access

□ Scope levels (from broadest to narrowest): Management Group > Subscription > Resource Group > Specific Resource

Fig 3.11 — Complete RBAC role reference for all Azure AI services

RBAC (Role-Based Access Control) — Concepts, Scope Levels & Common Roles

What is RBAC?

Role-Based Access Control is Azure's authorization system. A role assignment has exactly 3 parts:

WHO (principal): a user, group, service principal, or managed identity

WHAT (role): a named set of permissions, e.g. "Storage Blob Data Reader"

WHERE (scope): Management Group > Subscription > Resource Group > specific Resource

Scope Levels (broad – narrow)



□ Assign to "Specific Resource" for all app identities

Most Important RBAC Roles for AI Solutions

Service	Role Name	What it allows	Assign to
Azure OpenAI	Cognitive Services Ope	Call completions + embeddings (read-on	Your RAG API identit
Azure AI Search	Search Index Data Read	Query / search index	App that does search
Azure AI Search	Search Index Data Cont	Upsert + delete index documents	Ingestion pipeline i
Key Vault	Key Vault Secrets User	Read secret values	All application iden
Storage	Storage Blob Data Read	Download / read blobs	App that reads docum
Storage	Storage Blob Data Cont	Read + write blobs	Ingestion pipeline i
Container Regist	AcrPull	Pull container images from registry	AKS kubelet, Contain
Document Intelli	Cognitive Services Use	Analyse documents (extract text)	Ingestion function /
Service Bus	Azure Service Bus Data	Send messages to queues/topics	Producer application

Fig 3.12 — RBAC concepts: what it is, scope levels, most important roles

PART 3: NETWORKING

Chapter 4: VNet, Subnets, NSG & Private Endpoints

4.1 VNet Architecture

Azure VNet Architecture — Subnets, NSG, Private Endpoints, DNS

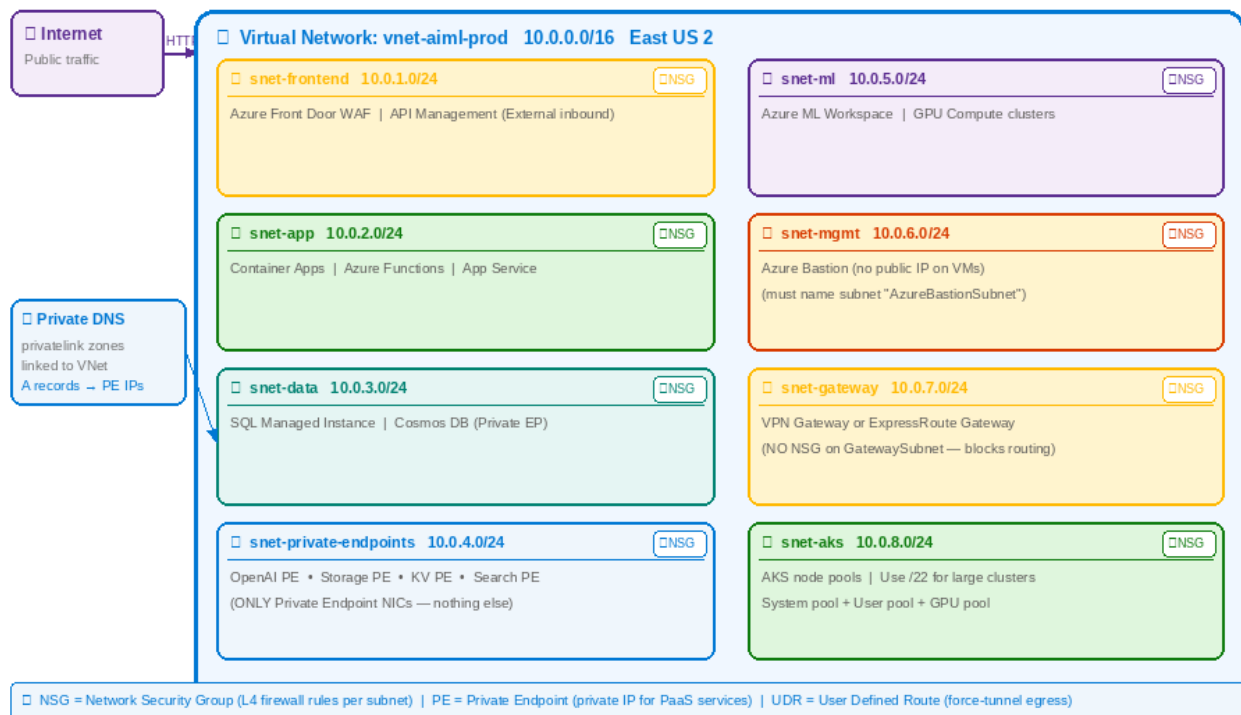


Fig 4.1 — VNet architecture: 8 subnets for the AI stack, NSG on each, Private DNS

Key Networking Terminology

VNet (Virtual Network)	Isolated software-defined network in Azure. You define the IP address space (e.g. 10.0.0.0/16 = 65,534 IPs). All resources inside share a private namespace and can communicate without internet.
Subnet	A subdivision of the VNet CIDR (e.g. 10.0.2.0/24 = 251 usable IPs). Resources are placed in subnets. NSGs are applied per subnet. Minimum /29 — Azure reserves 5 IPs per subnet.
NSG (Network Security Group)	Stateful Layer 4 firewall attached to a subnet or NIC. Rules specify source, destination, port, protocol, and allow/deny. Lower priority number = evaluated first. Default deny-all at 65500.
Service Tag	Named group of Azure service IP ranges maintained by Microsoft (e.g. "ApiManagement", "AzureMonitor", "Storage"). Use in NSG rules instead of hardcoding IP ranges that change.
Private Endpoint (PE)	A private IP address in YOUR VNet that maps to a specific PaaS service. Traffic to that IP stays entirely on the Azure backbone — never leaves your network. Requires a Private DNS Zone.
Private DNS Zone	An Azure DNS zone (e.g. privatelink.openai.azure.com) linked to your VNet. Resolves private endpoint FQDNs to their private IPs instead of public IPs. One zone per service type.
UDR (User Defined Route)	Custom route table entry that overrides Azure default routing. Common use: route 0.0.0.0/0 through Azure Firewall for egress inspection.

4.2 Create VNet — Portal Walkthrough

1	Navigate Portal → search "Virtual networks" → + Create → Basics: name=vnet-aiml-prod, region=East US 2
2	IP Addresses tab Address space: 10.0.0.0/16 → click "+ Add subnet" for each subnet below

Microsoft Azure

Home > Virtual networks > Create virtual network

Create virtual network

Basics

Security

IP addresses

Tags

Review + create

IP Addresses

Configure your VNet address space and subnets. Each subnet reserves IPs from the VNet CIDR.

IPv4 address space

10.0.0.0

/

16

= 65,534 usable IPs (10.0.0.1 – 10.0.255.254)

Subnets

+ Add subnet

Subnet name	Subnet address range	Usable IPs	Purpose
snet-frontend	10.0.1.0/24	251	Azure Front Door, API Management inbound
snet-app	10.0.2.0/24	251	Container Apps, Azure Functions
snet-data	10.0.3.0/24	251	SQL MI, Cosmos DB private endpoints
snet-private-endpoints	10.0.4.0/24	251	All PaaS private endpoints (OpenAI, Storage, KV, Search)
snet-ml	10.0.5.0/24	251	Azure ML Workspace, compute clusters
snet-mgmt	10.0.6.0/24	251	Azure Bastion, jump server
snet-gateway	10.0.7.0/24	251	VPN Gateway or ExpressRoute Gateway
snet-aks	10.0.8.0/24	251	AKS node pools (system + user pools)

Design rule: Never share a subnet between different workload tiers.

Reserve snet-private-endpoints exclusively for PE NICs — this makes NSG rules simple.

AKS nodes need at least /24 (251 IPs). For large clusters use /22 (1019 IPs).

Fig 4.2 — VNet IP Addresses tab: 8 subnets with CIDR ranges and purposes

Subnet	CIDR	Hosts here	NSG rule summary
snet-frontend	10.0.1.0/24	Front Door origin, API Management	Allow 443 inbound from Internet tag
snet-app	10.0.2.0/24	Container Apps, Functions, App Service	Allow 443 from ApiManagement only
snet-data	10.0.3.0/24	SQL MI, Cosmos DB private endpoints	Allow 1433/5432 from snet-app only
snet-private-endpoints	10.0.4.0/24	ONLY PE NICs (OpenAI, KV, Search, Storage)	No inbound needed — PE ignores NSG
snet-ml	10.0.5.0/24	Azure ML workspace, GPU compute	Allow 443 outbound to PE subnet
snet-mgmt	10.0.6.0/24	Azure Bastion (rename to AzureBastionSubnet)	Microsoft required rules
snet-gateway	10.0.7.0/24	VPN or ExpressRoute Gateway	NO NSG — blocks gateway routing
snet-aks	10.0.8.0/24	AKS node pools (use /22 for large)	AKS manages its own NSG

4.3 NSG Rules — Portal Walkthrough

NSG (Network Security Group) — What It Is and How to Configure Rules

What is an NSG?

A Network Security Group is a stateful Layer 4 firewall attached to a subnet or a network interface (NIC).

- Stateful: if you allow inbound port 443, Azure auto-allows the response traffic (no separate outbound rule)

- Rules evaluated by priority: lowest number = first

Service Tags — Use Instead of IP Ranges

AzureMonitor:	Azure Monitor health probes
ApiManagement:	API Management gateway
AzureLoadBalancer:	Internal Azure load balancer
AzureFrontDoor.Backend:	Front Door to your origin
VirtualNetwork:	All VNet address space
Internet:	Public internet
Storage:	All Azure Storage IDs

Recommended NSG rules for snet-app (application subnet)

Apply to: Container Apps, Functions, App Service subnet

Priority	Rule Name	Source → Destination	Port/Protocol	Action	Why
100	Allow_APIM_HTTPS	ApiManagement → snet-app	443/TCP	Allow	API Management reach
110	Allow_FrontDoor	AzureFrontDoor.Backend → sne	443/TCP	Allow	Front Door WAF route
200	Allow_to_PE_subnet	snet-app → snet-private-endp	443/TCP	Allow	Calls to OpenAI, KV,
300	Allow_AzureMonitor	AzureMonitor → snet-app	443/TCP	Allow	Health probes from m
310	Allow_LB_probe	AzureLoadBalancer → snet-app	*	Allow	Internal LB health p
400	Allow_VNet_internal	VirtualNetwork → VirtualNetw	*	Allow	Internal VNet-to-VNe
4000	Deny_all_inbound	Any → Any	*	Deny	Block everything not

Fig 4.3 — NSG concepts: what it is, service tags, recommended rules for snet-app

1	Create NSG Portal → Network security groups → + Create → name: nsg-app-aiml-prod → Create
2	Associate with subnet NSG → Subnets → Associate → vnet-aiml-prod → snet-app → OK
3	Add inbound rule NSG → Inbound security rules → + Add → fill all fields → Add

Microsoft Azure

Home > NSGs > nsg-app-aiml-prod > Inbound security rules > Add

Add inbound security rule

Source *	Destination *
Service Tag ☐	IP Addresses ☐
Source service tag *	Destination IP addresses / CIDR ranges *
ApiManagement ☐	10.0.2.0/24
Source port ranges *	Destination port ranges *
*	443
Protocol	Action
TCP ☐	Allow ☐
Priority *	Name *
200 (100-4096, lower = evaluated first)	Allow_APIM_to_App_443
Description	
Allow HTTPS from API Management service tag to app subnet. Required for APIM to reach backend.	
AddCancel	
☐ Priority guide: 100-199 = Critical allow rules 200-499 = Workload allow rules 500-999 = Monitoring/mgmt rules 4000-4096 = Deny rules (add explicit deny-all last) Deny-all rule: Add priority 4000, source=Any, dest=Any, port=*, action=Deny — catches anything not explicitly allowed above.	

Fig 4.4 — NSG Add inbound rule form: source service tag, destination CIDR, priority guide

⚠ Always add a Deny-all rule at priority 4000 (the highest number in your custom rules). This makes your allow-list explicit — anything not explicitly allowed is blocked. The default Azure deny-all is at 65500 and cannot be deleted, but adding one at 4000 makes your intent clear.

4.4 Private Endpoints — Complete Walkthrough

Private Endpoint — What It Is, How It Works, and Why You Need It

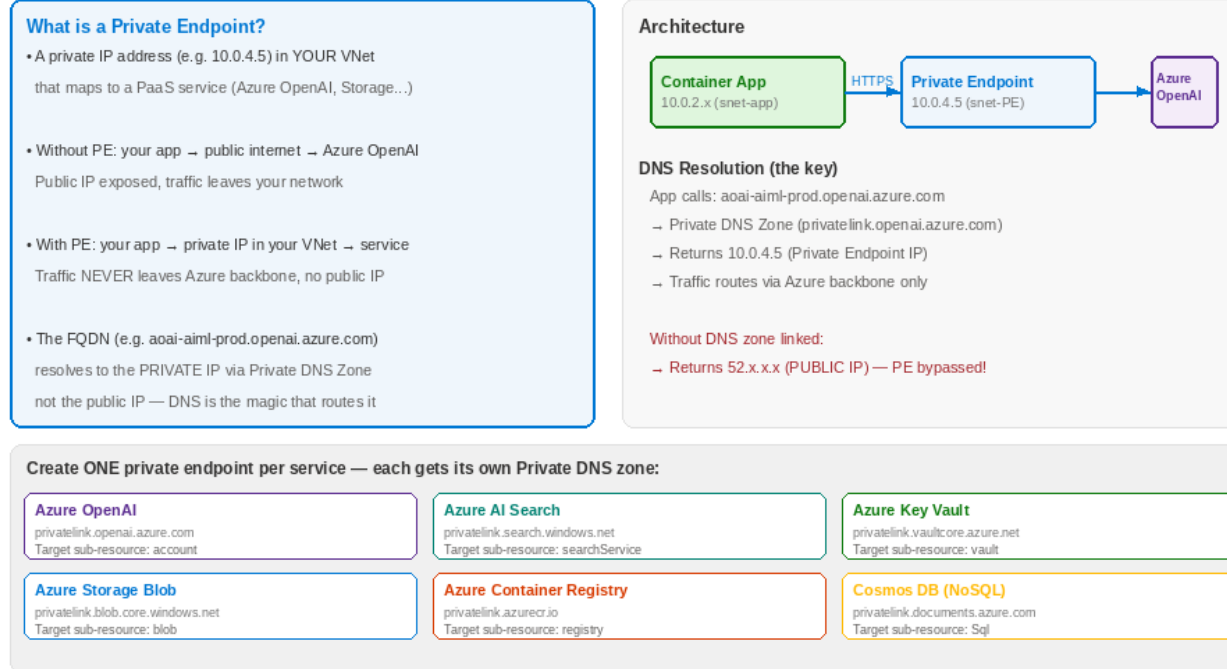


Fig 4.5 — What private endpoints are, how DNS resolution works, all service DNS zones

Private Endpoint Setup — Repeat for EACH service (OpenAI, Search, Storage, Key Vault, ACR)

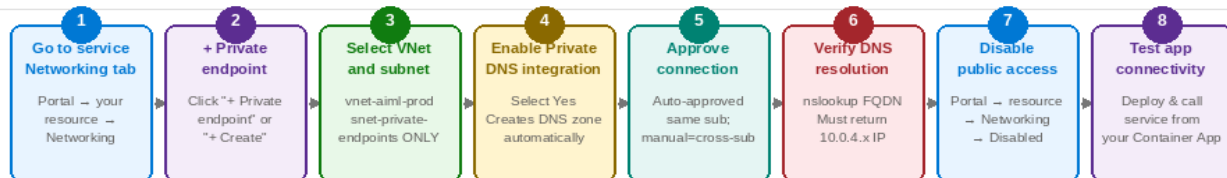


Fig 4.6 — Private endpoint setup workflow: 8 steps, repeat for each service

1	Open service Networking tab Portal → your OpenAI resource → Networking → Private endpoint connections → + Private endpoint																
	<table border="1" style="width: 100%; border-collapse: collapse;"> <thead> <tr> <th>PE creation field</th><th>Enter this</th><th>Why</th></tr> </thead> <tbody> <tr> <td>Name</td><td>pe-aoai-aiml-prod</td><td>Convention: pe-{service}-{project}-{env}</td></tr> <tr> <td>Subnet</td><td>snet-private-endpoints</td><td>ALWAYS use dedicated PE subnet — not snet-app</td></tr> <tr> <td>Target sub-resource</td><td>account (auto-selected)</td><td>Maps to the service endpoint you want private</td></tr> <tr> <td>Private DNS integration</td><td>Yes</td><td>Creates privatelink.openai.azure.com zone and A record automatically</td></tr> </tbody> </table>	PE creation field	Enter this	Why	Name	pe-aoai-aiml-prod	Convention: pe-{service}-{project}-{env}	Subnet	snet-private-endpoints	ALWAYS use dedicated PE subnet — not snet-app	Target sub-resource	account (auto-selected)	Maps to the service endpoint you want private	Private DNS integration	Yes	Creates privatelink.openai.azure.com zone and A record automatically	
PE creation field	Enter this	Why															
Name	pe-aoai-aiml-prod	Convention: pe-{service}-{project}-{env}															
Subnet	snet-private-endpoints	ALWAYS use dedicated PE subnet — not snet-app															
Target sub-resource	account (auto-selected)	Maps to the service endpoint you want private															
Private DNS integration	Yes	Creates privatelink.openai.azure.com zone and A record automatically															
2	Verify DNS zone Portal → Private DNS zones → privatelink.openai.azure.com → Virtual network links → confirm VNet listed																

Microsoft Azure

Home > Private DNS zones > privatelink.openai.azure.com > Virtual network links

Virtual network links

[privatelink.openai.azure.com](#) | [Private DNS Zone](#) | [Record sets](#) | [Virtual network links](#) | [Access control \(IAM\)](#)

Why this matters

Without this link, resources in your VNet will resolve the OpenAI FQDN to its PUBLIC IP — bypassing the private endpoint.
With the VNet link + A record: nslookup aoai-aiml-prod.openai.azure.com → 10.0.4.5 (your private endpoint IP).

+ Add

Link name	Virtual network	Auto registration	Status
vnet-aiml-prod-link	vnet-aiml-prod (rg-aiml-prod)	Disabled (recommended)	Completed
vnet-dev-link	vnet-aiml-dev (rg-aiml-dev)	Disabled	Completed

Record sets in this zone

Name	Type	TTL	Value (Private IP)
aoai-aiml-prod	A	3600	10.0.4.5 ← this is the private endpoint NIC IP
aoai-aiml-dev	A	3600	10.1.4.8

Auto registration = Disabled: Do NOT enable auto-registration for private endpoint zones.
Auto-registration is for VMs registering their own hostnames — not for PaaS service records.
The A record must be created manually pointing to the PE NIC private IP address.

One private DNS zone per service type — do not reuse zones:
OpenAI → privatelink.openai.azure.com | Storage Blob → privatelink.blob.core.windows.net
Key Vault → privatelink.vaultcore.azure.net | AI Search → privatelink.search.windows.net

Fig 4.7 — Private DNS Zone: VNet link and A record pointing to private endpoint IP

3 Test from inside VNet

Open Cloud Shell → nslookup aoai-aiml-prod.openai.azure.com → must return 10.0.4.x, NOT a public IP

```
# Test private endpoint from Cloud Shell
nslookup aoai-aiml-prod.openai.azure.com

# CORRECT output (PE working):
# Name:      aoai-aiml-prod.openai.azure.com
# Address: 10.0.4.5 ← private IP from snet-private-endpoints

# WRONG output (PE NOT working — DNS zone not linked):
# Name:      aoai-aiml-prod.openai.azure.com
# Address: 52.xxx.xxx.xxx ← public IP, traffic goes over internet!

# If you see public IP, check:
# 1. Private DNS zone privatelink.openai.azure.com exists
# 2. VNet link to vnet-aiml-prod exists in that DNS zone
# 3. A record in DNS zone matches PE NIC private IP

# Get PE NIC IP:
az network nic show --ids \
  $(az network private-endpoint show -n pe-aoai-aiml-prod -g rg-aiml-prod \
    --query networkInterfaces[0].id -o tsv) \
  --query ipConfigurations[0].privateIPAddress -o tsv
```

4 Disable public access

Portal → OpenAI resource → Networking → select "Disabled" → Save. Repeat for each service.

PART 4: AZURE SERVICES — SETUP, CONFIGURE & USE

Chapter 5: Azure Key Vault — Zero-Secret Pattern

Microsoft Azure

Home > Key vaults > Create key vault

Create a key vault

Basics

Access configuration

Networking

Tags

Review + create

Permission model

☐ Vault access policy (legacy)
Resource-level allow list per principal. Works but hard to audit at scale. Avoid for new vaults.

☒ Azure role-based access control (RBAC) — Recommended
Uses standard Azure RBAC roles. Auditable, consistent with all other Azure services. Choose this.

Resource access

☐ Azure Virtual Machines for deployment
Allows VMs to retrieve certificates from Key Vault

☐ Azure Resource Manager for template deployment
Allows ARM deployments to retrieve secrets

☐ Azure Disk Encryption for volume encryption
Allows ADE to retrieve keys for disk encryption

☐ Soft-delete and purge protection are enabled by default and CANNOT be disabled after creation.
Soft-delete: deleted secrets retained for 90 days (recoverable). Purge protection: even admins cannot permanently delete until retention period expires. Both required for com

After creation — add secrets:

Secrets > Generate/Import

Name *

Value *

openai-endpoint

https://aoai-aiml-prod.openai.azure.com/

Fig 5.1 — Key Vault Access Configuration: RBAC model is required, never legacy policy

Azure Key Vault — Zero-secret Pattern: Managed Identity → RBAC → Key Vault

```

graph LR
    App[App  
(Container App  
AKS / Function)] -- uses --> MI[Managed  
Identity  
(system-assigned)]
    MI -- authenticates with --> EntraID[Entra ID  
(token exchange  
OAuth 2.0 flow)]
    EntraID -- token granted --> KV_RBAC[Key Vault  
RBAC  
(Key Vault Secrets  
User role)]
    KV_RBAC -- reads secret --> KV[Key Vault  
(secret /  
certificate / key)]
    KV -.->|secret value returned in memory (never logged)| App
  
```

❌ WRONG: AOAI_KEY = "sk-abc..." in .env file | env var OPENAI_KEY hardcoded | secret committed to git

```

from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential

# No API key anywhere — credential comes from managed identity
client = SecretClient("https://kv-aiml-prod.vault.azure.net/", DefaultAzureCredential())
openai_endpoint = client.get_secret("openai-endpoint").value
# Use endpoint with managed identity — still no key needed!
  
```

Fig 5.2 — Zero-secret pattern: app MI → KV secret read → value never in code

```

# Create KV with RBAC model, add secrets, use KV references in apps
az keyvault create -n kv-aiml-prod -g rg-aiml-prod \
  --enable-rbac-authorization --enable-purge-protection
  
```

Azure AI/ML Practitioner Handbook | portal.azure.com · ai.azure.com | Python & CLI Examples Throughout

```
# Store secrets
az keyvault secret set --vault-name kv-aiml-prod \
    --name "openai-endpoint" --value "https://aoai-aiml-prod.openai.azure.com/"

# Container App uses KV references – app reads as normal env var
az containerapp update -n rag-api -g rg-aiml-prod --set-env-vars \
    OPENAI_ENDPOINT="@Microsoft.KeyVault(SecretUri=https://kv-aiml-prod.vault.azure.net/secrets/
openai-endpoint/)"

# Python: read directly when you need dynamic config
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
kv = SecretClient("https://kv-aiml-prod.vault.azure.net/", DefaultAzureCredential())
value = kv.get_secret("openai-endpoint").value
```

Chapter 6: Azure Container Registry (ACR)

Home > Container registries > acraimlprod

acraimlprod

Container registry | East US 2 | rg-aiml-prod | Premium

Overview Access control (IAM) Networking Repositories Webhooks Replications Tasks

Essentials

Login server	acraimlprod.azurecr.io
SKU	Premium
Status	Ready
Location	East US 2
Admin user	Disabled (use MI / AcrPull role)
Public network access	Disabled (private EP only)
Zone redundancy	Enabled
Encryption	Microsoft-managed keys
Geo-replication	West Europe (1 replica)
Retention policy	30 days

Repositories

Repository	Tags	Last push
rag-api	12	10 min ago
doc-processor	8	2 hrs ago
ml-inference	5	Yesterday
admin-portal	3	3 days ago

ACR Security Best Practices (apply these after creation):

- Admin user DISABLED — use managed identity + AcrPull RBAC role (never share admin credentials)
- Public network access DISABLED — create private endpoint in snet-private-endpoints
- Use ACR Tasks (az acr build) to build images without local Docker daemon

```
az acr build --registry acraimlprod --image rag-api:$(git rev-parse --short HEAD) .
```

Fig 6.1 — ACR overview: login server, SKU (Premium), geo-replication, security settings

```
# Create Premium ACR with private endpoint
az acr create -n acraimlprod -g rg-aiml-prod --sku Premium --admin-enabled false

# Build image in ACR (no local Docker needed)
az acr build --registry acraimlprod \
  --image rag-api:$(git rev-parse --short HEAD) \
  --image rag-api:latest .

# Grant Container App MI the AcrPull role
az role assignment create --role AcrPull \
  --assignee $(az containerapp show -n rag-api -g rg-aiml-prod --query identity.principalId -o tsv) \
  --scope $(az acr show -n acraimlprod -g rg-aiml-prod --query id -o tsv)
```

Chapter 7: Container Apps Environment & Container Apps

Microsoft Azure

Home > Container Apps Environments > cae-aiml-prod

cae-aiml-prod

Container Apps environment | East US 2 | rg-aiml-prod

Overview Apps Networking Monitoring Settings

Environment details

Type	Workload profiles (serverless)
Virtual IP	Internal (VNet-integrated)
Subnet	snet-app (10.0.2.0/24)
Infra subnet	10.0.8.0/24 (managed)
Zone redundancy	Enabled (3 AZs)
Custom DNS	168.63.129.16 (Azure default)
Workload profiles	Consumption (auto-scale)
Log Analytics	law-aiml-prod (connected)
Apps running	4 container apps
Public IP	None — internal VNet only

Container Apps in this environment

App name	Replicas	Ingress	Status
rag-api	2 / max 50	External HTTPS	Running
doc-processor	0 / max 20	Internal	Running
agent-service	1 / max 10	Internal	Running
admin-portal	1 / max 5	External HTTPS	Running

☐ Container Apps Environment = the shared infrastructure layer hosting multiple Container Apps.
VNet-integrated: all apps in this env share the snet-app subnet — they communicate internally without internet.
Zone redundancy: replicas spread across 3 AZs — one AZ failure does not take down your app.
Log Analytics: all stdout/stderr from all apps goes to the same workspace for unified querying.

Fig 7.1 — Container Apps Environment: VNet-integrated, zone-redundant, shared infrastructure

Microsoft Azure

Home > Container Apps > Create > Basics

Create Container App

Basics **Container** Bindings Ingress Tags Review + create

Container details

☐ Use quickstart image
Uncheck this to use your own container image from ACR

Image source *
Azure Container Registry ☐

Registry *
acraimlprod (acraimlprod.azurecr.io) ☐

Image tag *
latest ☐

In production: use a specific SHA tag (not "latest") for reproducibility

Image *
rag-api ☐

Resource allocation per replica

CPU cores
1.0 Range: 0.25 – 4.0 vCPU per replica

Memory (GiB)
2.0 Range: 0.5 – 8.0 GiB per replica. Must be 2× CPU.

Environment variables

[+ Add](#)

Name	Source	Value
------	--------	-------

Fig 7.2 — Container App creation: image source, resource allocation, Key Vault secret refs

Microsoft Azure

Home > Container Apps > rag-api > Scale and replicas

rag-api | Container App

Overview

Deployments

Ingress

Scale and replicas

Identity

Replica count

Min replicas

1

(set 0 for dev; 1+ for prod to avoid cold starts)

Max replicas

50

(cost ceiling; tune based on load testing)

Scale rules

KEDA scalers trigger scale-out events based on external metrics.

+ Add

+ Add scale rule

Rule name *

http-scale-rule

Scale rule type *

HTTP scaling

Concurrent requests *

10

Scale out when concurrent HTTP requests per replica > 10

— OR add Service Bus scale rule —

Scale rule type

Azure Service Bus

Queue / topic name

doc-process-queue

KEDA scale rules: HTTP = scale on concurrent requests | Azure Service Bus = scale on queue depth

Azure Monitor = scale on any Azure metric | Cron = schedule-based (e.g. pre-warm every morning at 8am)

Cool-down: Container Apps waits 300s (5 min) after last event before scaling in — prevents thrashing.

Fig 7.3 — Scale rules: min/max replicas, HTTP KEDA scaler, Service Bus queue scaler

Container App Deployment — Code to Production via GitHub Actions + Canary

1

Code pushed to GitHub

PR merged to main branch triggers Actions

2

Run tests & lint

pytest coverage
mypy type check
ruff lint

3

Build image in ACR

az acr build
no local Docker
multi-stage

4

Scan for vulnerabilities

Defender for Containers
block critical

5

Deploy to staging

100% traffic
Run smoke tests
verify health

6

Manual approval

GitHub env
protection rule
team approval

7

Canary 10% traffic

New revision
10% weight
5 min monitor

8

Full cutover

100% to new
old revision
retained for rollback

Fig 7.4 — Deployment workflow: code → ACR → staging → manual approval → canary → production

```
# Create CAE with VNet integration
az containerapp env create -n cae-aiml-prod -g rg-aiml-prod \
  --location eastus2 \
  --infrastructure-subnet-resource-id $(az network vnet subnet show \
    -n snet-app --vnet-name vnet-aiml-prod -g rg-aiml-prod --query id -o tsv)

# Create Container App with MI, KV refs, KEDA scale rule
az containerapp create -n rag-api -g rg-aiml-prod \
  --environment cae-aiml-prod \
  --image acraimlprod.azurecr.io/rag-api:latest \
  --target-port 8000 --ingress external \
  --min-replicas 1 --max-replicas 50 \
  --scale-rule-name http-rule --scale-rule-http-concurrency 10 \
  --cpu 1.0 --memory 2.0Gi \
  --system-assigned \
  --set-env-vars ENVIRONMENT=production \
  OPENAI_ENDPOINT="@Microsoft.KeyVault(SecretUri=https://kv-aiml-prod.vault.azure.net/secrets/openai-endpoint/)"
```

Azure AI/ML Practitioner Handbook | portal.azure.com · ai.azure.com | Python & CLI Examples Throughout

Chapter 8: App Service & Azure Functions

The screenshot displays the Azure portal interface for an App Service named 'admin-portal-aiml'. The breadcrumb navigation shows 'Home > App Services > admin-portal-aiml'. The main header includes 'App Service Overview', 'East US 2', 'rg-aiml-prod', 'B2 (Basic)', 'Deployment', 'Configuration', 'Scale out', 'TLS/SSL', 'Networking', and 'Identity'. Below the header, there are three tabs: 'Running' (selected), 'Public', and 'Python 3.11'. The 'Essentials' section on the left lists various settings: URL (https://admin-portal-aiml.azurewebsites.net), App Service plan (asp-aiml-prod (B2 = 2 vCPU, 3.5GB RAM)), OS (Linux), Runtime (Python 3.11), Managed identity (System-assigned (enabled)), Deployment (Continuous deployment), GitHub (main branch), Custom domain (None), Health check (/health — 30s interval). The 'Configuration — App Settings' section on the right shows environment variables for the app, including OPENAI_ENDPOINT, SEARCH_ENDPOINT, APPLICATIONINSIGHTS_..., ENVIRONMENT, and SCM_DO_BUILD_DURING_DEPLOYMENT. A note at the bottom explains the difference between App Service and Container Apps, and provides instructions on how to use App Service, scale out, and deploy.

Essentials

- URL: https://admin-portal-aiml.azurewebsites.net
- App Service plan: asp-aiml-prod (B2 = 2 vCPU, 3.5GB RAM)
- OS: Linux
- Runtime: Python 3.11
- Managed identity: System-assigned (enabled)
- Deployment: Continuous deployment
- GitHub (main branch)
- Custom domain: None (use Front Door for custom domain)
- Health check: /health — 30s interval

Configuration — App Settings
(Environment variables for your app)

Name	Value	Source
OPENAI_ENDPOINT	@Microsoft.KeyVault(SecretUr	KV ref
SEARCH_ENDPOINT	@Microsoft.KeyVault(SecretUr	KV ref
APPLICATIONINSIGHTS_...	@Microsoft.KeyVault(...)	KV ref
ENVIRONMENT	production	Manual
SCM_DO_BUILD_DURING_DEPLOYMENT		Manual

□ App Service vs Container Apps: App Service is PaaS for web apps/APIs; Container Apps is for containerised apps. When to use App Service: traditional web frameworks (Django, Flask, FastAPI) without containerisation. Scale out (horizontal): App Service plan → Scale out → Custom autoscale → add CPU-based rule. Deployment slots: create staging slot → deploy → swap with production (zero downtime deployment).

Fig 8.1 — App Service: URL, plan, runtime, MI, deployment settings, app configuration

The screenshot displays the Azure portal interface for a Function App named 'fn-aiml-proc'. The breadcrumb navigation shows 'Home > Function App > fn-aiml-proc'. The main header includes 'Function App Overview', 'East US 2', 'rg-aiml-prod', 'Python 3.11', 'Functions', 'App keys', 'Configuration', 'Identity', and 'Networking'. Below the header, there are two tabs: 'Running' (selected) and 'Flex Consumption'. The 'Essentials' section on the left lists various settings: Runtime (Python 3.11), Hosting plan (Flex Consumption (scale to 0)), Storage account (staimlprod (required by Functions)), Managed identity (System-assigned (enabled)), VNet integration (snet-app (outbound via VNet)), App Insights (appi-aiml-prod (connected)), Max instances (100 (Flex Consumption)), and Function timeout (10 minutes (for long AI calls)). The 'Functions in this app' section on the right shows a list of functions with their names, trigger types, and last run times. A note at the bottom explains the difference between Consumption and Flex Consumption hosting plans, and provides instructions on how to use App Service, scale out, and deploy.

Essentials

- Runtime: Python 3.11
- Hosting plan: Flex Consumption (scale to 0)
- Storage account: staimlprod (required by Functions)
- Managed identity: System-assigned (enabled)
- VNet integration: snet-app (outbound via VNet)
- App Insights: appi-aiml-prod (connected)
- Max instances: 100 (Flex Consumption)
- Function timeout: 10 minutes (for long AI calls)

Functions in this app

Function name	Trigger type	Last run
ingest_document	Blob trigger (Event Grid)	2 min ago □
process_queue	Service Bus trigger	5 min ago □
scheduled_embed	Timer (daily 2am)	12 hrs ago □
http_ingest	HTTP trigger (internal)	Just now □

□ Hosting plans: Consumption (max 5min timeout, cold start) | Flex Consumption (up to 60min, scale to 0, VNet) Premium plan: pre-warmed workers (no cold start), VNet, up to 60min, GPU support. Always enable VNet integration for production — functions must reach private endpoints for OpenAI, Search, KV. KV references: set OPENAI_ENDPOINT="@Microsoft.KeyVault(SecretUri=...)" in App Settings — never plain-text values.

Fig 8.2 — Function App: functions list, trigger types, hosting plan, VNet integration

Azure Functions — Trigger Types, Bindings, Hosting Plans & Patterns

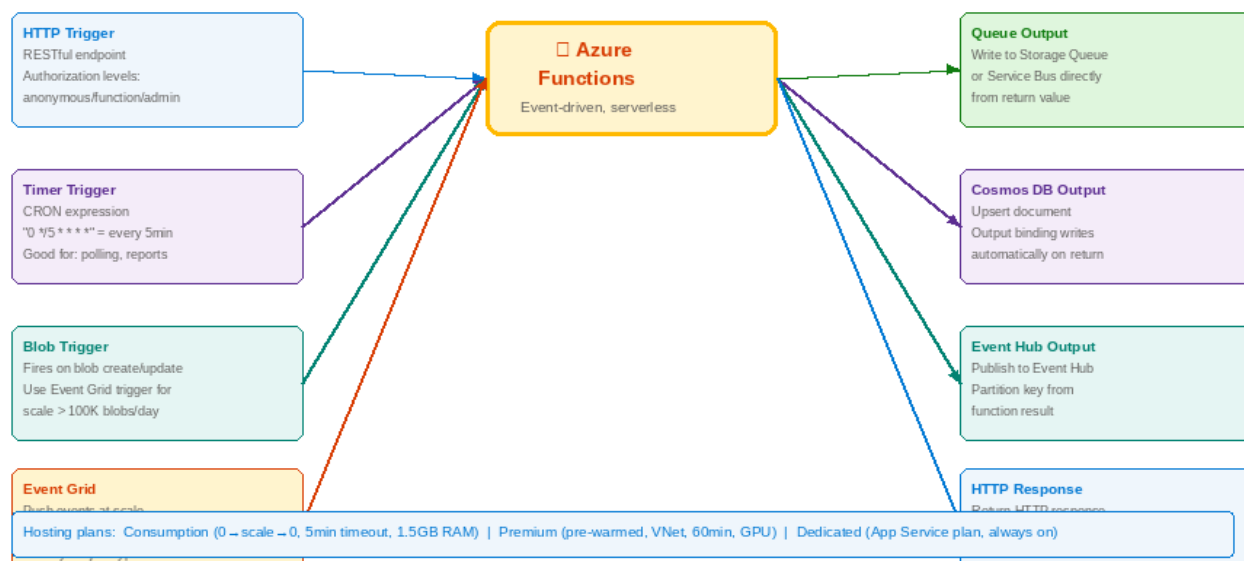


Fig 8.3 — Function trigger types: HTTP, Timer, Blob, Event Grid, Service Bus — all shown

```
# App Service — Python web app
az appservice plan create -n asp-aiml-prod -g rg-aiml-prod --sku B2 --is-linux
az webapp create -n admin-portal-aiml -g rg-aiml-prod -p asp-aiml-prod --runtime "PYTHON:3.11"
az webapp identity assign -n admin-portal-aiml -g rg-aiml-prod

# Azure Functions — Flex Consumption (scale to 0, VNet access)
az functionapp create -n fn-aiml-proc -g rg-aiml-prod \
  --storage-account staimlprod \
  --flexconsumption-location eastus2 \
  --runtime python --runtime-version 3.11 \
  --assign-identity "[system]"

# Function triggered by new blob in Storage — ingestion pipeline
import azure.functions as func
app = func.FunctionApp()

@app.blob_trigger(arg_name="blob", path="raw-docs/{name}",
                  connection="AzureWebJobsStorage")
def ingest_document(blob: func.InputStream) -> None:
    # extract text → chunk → embed → upsert to AI Search
    pass
```

Chapter 9: Storage Account — Blob, ADLS Gen2 & Lifecycle

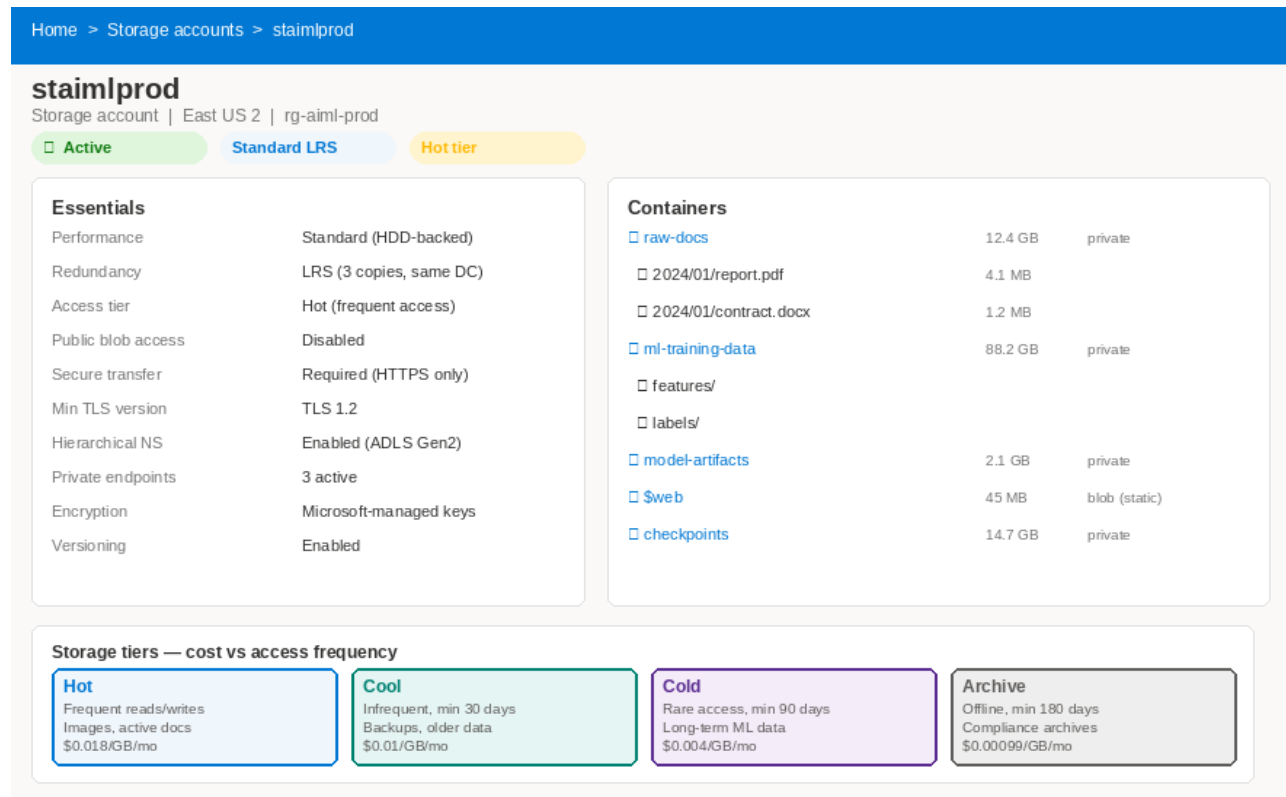


Fig 9.1 — Storage account: properties, container hierarchy, access tier comparison

```
# Create with ADLS Gen2 (hierarchical namespace) + security hardening
az storage account create -n staimlprod -g rg-aiml-prod -l eastus2 \
  --sku Standard_ZRS --kind StorageV2 \
  --enable-hierarchical-namespace true \
  --allow-blob-public-access false --min-tls-version TLS1_2 \
  --public-network-access Disabled

# Python SDK – upload, download, list (keyless auth)
from azure.storage.blob import BlobServiceClient
from azure.identity import DefaultAzureCredential

client = BlobServiceClient("https://staimlprod.blob.core.windows.net",
                          DefaultAzureCredential())

blob = client.get_blob_client("raw-docs", "2024/q4-report.pdf")
with open("q4.pdf", "rb") as f:
    blob.upload_blob(f, overwrite=True)

data = blob.download_blob().readall()

for b in client.get_container_client("raw-docs").list_blobs(name_starts_with="2024/"):
    print(f"{b.name}: {b.size:,} bytes, modified: {b.last_modified}")
```

Chapter 10: API Management

Microsoft Azure

□ □ □

Home > API Management services > apim-aiml-prod > APIs > RAG API

RAG API — Policy Editor

apim-aiml-prod | East US 2 | Standard tier
APIs Products Subscriptions Policies Named values Analytics

```
All APIs — Inbound Policy (applies to every request)
<policies>
  <inbound>
    <!-- 1: Validate Entra ID JWT on every request -->
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401">
      <openid-config url="https://login.microsoftonline.com/{tenant-id}/v2.0/.well-known/openid-configuration">
        <audiences><audience>api://rag-api-client-id</audience></audiences>
      </validate-jwt>
    <!-- 2: Rate limit: 100 calls / 60 sec per user (prevents abuse) -->
    <rate-limit-by-key calls="100" renewal-period="60"
      counter-key="@((context.Request.Headers['Authorization']).Substring(7))" />
    <!-- 3: Set correlation ID for distributed tracing -->
    <set-header name="X-Correlation-Id" exists-action="skip">
      <value>@(Guid.NewGuid().ToString())</value></set-header>
    <!-- 4: Forward managed identity token to backend (no secret in policy) -->
    <authentication-managed-identity resource="https://rag-api-backend.internal.net"/>
  </inbound>
  <backend><forward-request timeout="30"/></backend>
  <on-error><return-response><set-status code="@((int)context.LastError.StatusCode)"/>
    <set-body>@("{"error":" "+context.LastError.Message+"}")</set-body></return-response></on-error>
</policies>
```

Validates Entra ID Bearer token — ensures only authenticated users call your AI API

100 requests per 60 seconds per user — prevents one user exhausting your OpenAI quota

Injects a UUID in X-Correlation-Id header — links APIM logs to App Insights traces

APIM uses its own MI to authenticate to your backend Container App — no hardcoded token

Fig 10.1 — APIM policy editor: JWT validation, rate limiting, correlation ID, MI forwarding

```
# APIM all-APIs inbound policy — paste in portal: APIM → APIs → All operations → Policy
<policies>
  <inbound>
    <!-- 1: Validate Entra ID JWT -->
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401">
      <openid-config url="https://login.microsoftonline.com/{tenant-id}/v2.0/.well-known/openid-configuration">
        <audiences><audience>api://rag-api-client-id</audience></audiences>
      </validate-jwt>
    <!-- 2: Rate limit 100/min per user -->
    <rate-limit-by-key calls="100" renewal-period="60"
      counter-key="@((context.Request.Headers['Authorization']).Last().Substring(7))"/>
    <!-- 3: Correlation ID for distributed tracing -->
    <set-header name="X-Correlation-Id" exists-action="skip">
      <value>@(Guid.NewGuid().ToString())</value></set-header>
  </inbound>
  <backend><forward-request timeout="30"/></backend>
</policies>
```

Chapter 11: Azure AI Search

Microsoft Azure

Home > Azure AI Search > Create search service

Create a search service

Basics

Scale

Networking

Tags

Review + create

Subscription *

Pay-As-You-Go

Resource group *

rg-aiml-prod

Service name **

aiml-search-prod

Location **

(US) East US 2

Globally unique URL: aiml-search-prod.search.windows.net

Pricing tier *

(click Change tier to see all options)

F	Free	50 MB, 3 indexes, no SLA	Dev/test only
B	Basic	2 GB, 15 indexes, 1 replica, no semantic	Light workloads
S1	Standard S1	25 GB/partition, semantic ranking, SLA	— Production RAG — minimum recommended
S2	Standard S2	100 GB/partition, more replicas	Large corpora
L1	Storage Optimized L1	2 TB/partition, lower QPS	Huge datasets

☐ Semantic ranking requires Standard tier or above. Enable it under: Settings > Semantic ranker.
Replicas: 2+ replicas = 99.9% SLA. 3+ replicas = read-heavy HA.
Partitions: Each partition = more storage + compute. Add partitions for large indexes.
Scale tab: Set replicas=2, partitions=1 minimum for production.

Fig 11.1 — AI Search creation: tier comparison (S1 minimum), replicas for SLA

Microsoft Azure

Home > AI Search > aiml-search-prod > Search explorer

Search explorer

Test your index directly from the portal without writing code

Index:

rag-docs-index

API version:

2024-07-01

Query type:

Hybrid

Query (JSON body):

Search

{"search": "What is our refund policy?", "top": 3, "queryType": "semantic", "semanticConfiguration": "default"}

Results (3 documents, 48ms)

@search.score: 0.94 | @search.rerankerScore: 2.84

id: refund-policy_0 | source: https://blob.../policies/refund.pdf

Customers may return items within 30 days of purchase for a full refund.....

@search.score: 0.87 | @search.rerankerScore: 2.41

id: refund-policy_2 | source: https://blob.../policies/refund.pdf

Digital downloads and software licenses are non-refundable once activated.....

@search.score: 0.71 | @search.rerankerScore: 1.93

id: faq_12 | source: https://blob.../docs/faq.pdf

Q: How do I request a refund? A: Contact support@company.com with your order number.....

☐ Search Explorer lets you test queries without code. Check "@search.score" and "@search.rerankerScore".
Higher rerankerScore = better semantic match. First result should be most relevant. If not, adjust your index config.

Fig 11.2 — Search Explorer: test hybrid queries, relevance scores, validate index

```
# Create index + hybrid query (production RAG pattern)
from azure.search.documents.indexes import SearchIndexClient
from azure.search.documents.indexes.models import (
```

Azure AI/ML Practitioner Handbook | portal.azure.com · ai.azure.com | Python & CLI Examples Throughout

```
SearchIndex, SimpleField, SearchableField, SearchField,
VectorSearch, HnswAlgorithmConfiguration, VectorSearchProfile,
SemanticConfiguration, SemanticSearch, SemanticPrioritizedFields, SemanticField)
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

# Create index
index = SearchIndex(name="rag-docs-index", fields=[
    SimpleField("id", "Edm.String", key=True, filterable=True),
    SearchableField("content", "Edm.String", analyzer_name="en.microsoft"),
    SimpleField("source_url", "Edm.String", filterable=True, retrievable=True),
    SimpleField("tenant_id", "Edm.String", filterable=True),
    SearchField("embedding", "Collection(Edm.Single)",
        vector_search_dimensions=3072, vector_search_profile_name="hnsw"),
], vector_search=VectorSearch(
    profiles=[VectorSearchProfile("hnsw", "hnsw-algo")],
    algorithms=[HnswAlgorithmConfiguration("hnsw-algo",
        parameters={"m": 4, "efConstruction": 400, "metric": "cosine"})]),
    semantic_search=SemanticSearch(configurations=[
        SemanticConfiguration("default", SemanticPrioritizedFields(
            content_fields=[SemanticField("content")]))]))))

SearchIndexClient(ENDPOINT, cred).create_or_update_index(index)

# Hybrid search (BM25 + vector + semantic reranker)
results = SearchClient(ENDPOINT, "rag-docs-index", cred).search(
    search_text=question,
    vector_queries=[VectorizedQuery(vector=q_embedding, k_nearest_neighbors=5,
    fields="embedding")],
    filter=f"tenant_id eq '{tenant_id}'", # ALWAYS filter by tenant for data isolation
    query_type="semantic", semantic_configuration_name="default", top=5)
```

Chapter 12: Azure AI Foundry

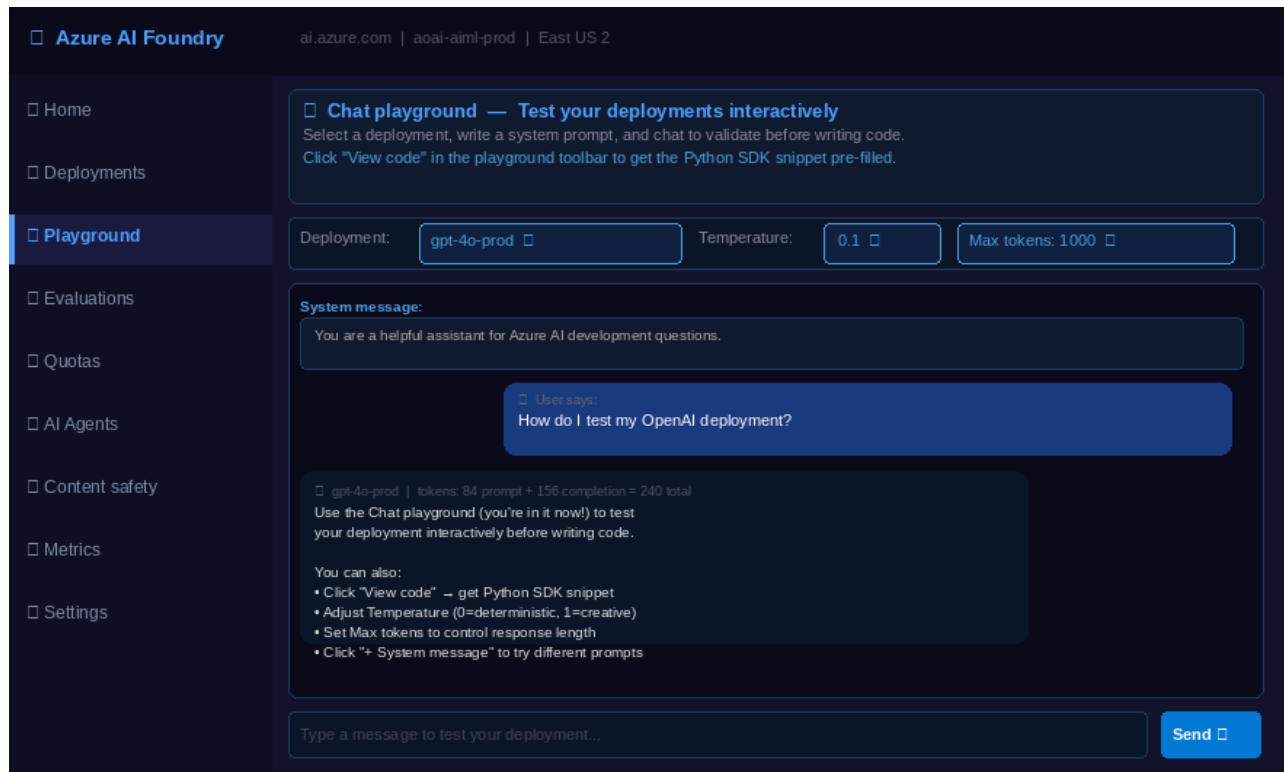


Fig 12.1 — AI Foundry playground: test deployments, see token counts, get Python code

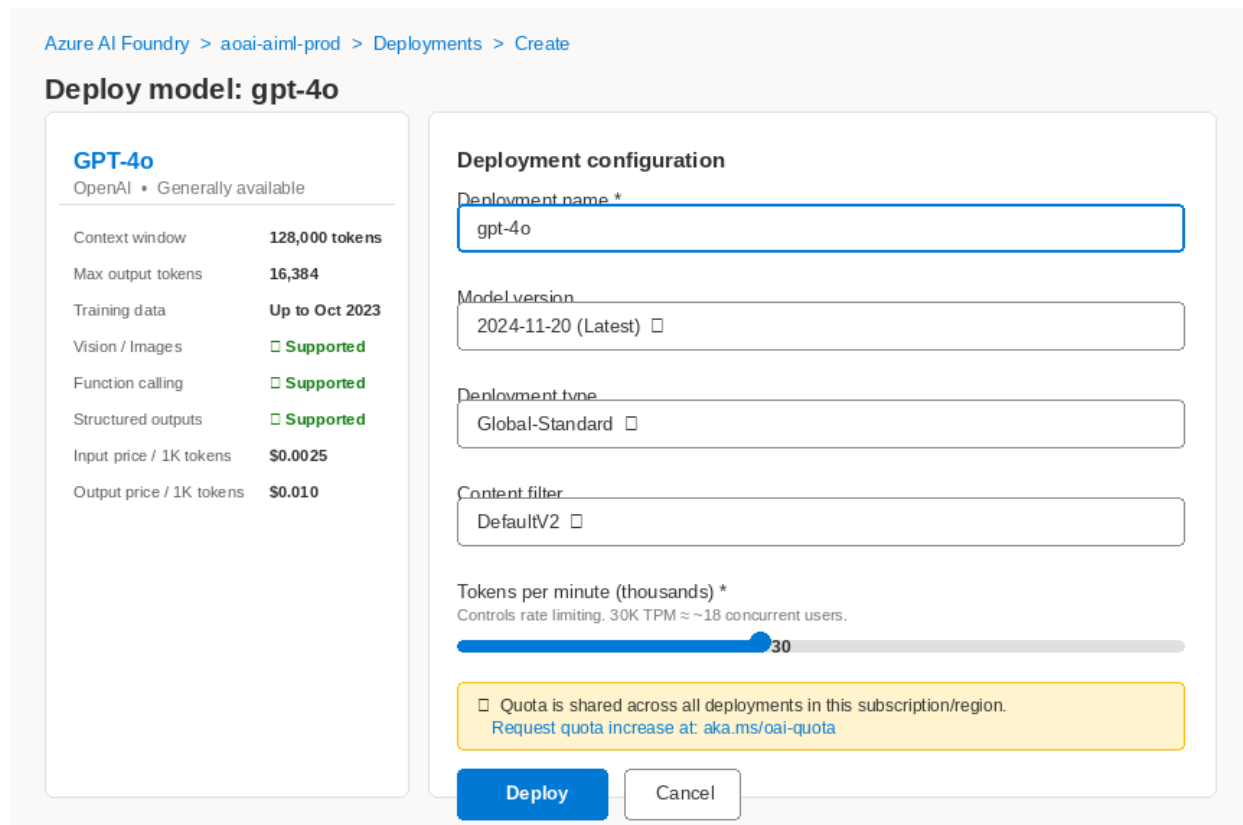


Fig 12.2 — Model deployment: Global-Standard type, content filter, TPM quota

Setting	Value	Why
Deployment name	gpt-4o-prod	Your CODE references this name, not the model name.
Model	gpt-4o	Best quality. Use gpt-4o-mini for classification (5x cheaper).
Deployment type	Global-Standard	Fewer 429 errors — uses global spare capacity.
Content filter	DefaultV2	Blocks harmful categories. Create custom profiles for specialized domains.
TPM quota	30K to start	Request increase: AI Foundry → Quotas → deployment → Request.

Chapter 13: Log Analytics Workspace & Application Insights

Microsoft Azure

Home > Log Analytics workspaces > law-ai-ml-prod > Logs

Logs — law-ai-ml-prod

KQL query editor — paste and run queries below

Overview **Logs** Workbooks Alerts Usage and estimated costs

Kusto Query Language (KQL)

```
// Query 1: Average OpenAI latency by hour (last 24h)
AppDependencies
| where TimeGenerated > ago(24h)
| where Target contains "openai.azure.com"
| summarize avg_ms=avg(DurationMs), p95=percentile(DurationMs,95), calls=count()
  by bin(TimeGenerated,1h)
| render timechart

// Query 2: Error rate per endpoint
AppRequests | where TimeGenerated > ago(1h)
| summarize errors=countif(Success==false), total=count() by Name
| extend error_pct = round(errors*100.0/total,2)
```

Run Query

Results (sample — last hour)

TimeGenerated	avg_ms	p95	calls
2024-11-20 10:00	1,234	2,847	1,284
2024-11-20 11:00	987	1,923	1,847

Key Log Analytics tables for Azure AI workloads:

AppRequests	Inbound HTTP requests to your Container App/Function/App Service
AppDependencies	Outbound calls your app makes (OpenAI, AI Search, SQL, Service Bus...)
AppExceptions	Unhandled exceptions with full stack trace
AppTraces	logging.info/warning/error output from your Python code
customMetrics	Custom metrics: token counts, RAG latency, chunk counts via OTel SDK
ContainerAppConsoleLogs_CL	stdout/stderr from Container App containers

Fig 13.1 — Log Analytics: KQL editor with sample queries and table reference

Tracing & Logging Pattern — OpenTelemetry + Azure Monitor SDK

Traces

Distributed request flow
Operation ID links all spans
Parent-child span hierarchy
Latency at each hop
→ AppDependencies table
→ End-to-end waterfall

Metrics

Numeric measurements
Request counts, latency
Token usage, error rates
15-second granularity
Custom metrics SDK
→ customMetrics table

Logs

Structured key-value records
Correlated by operationId
Log levels: DEBUG – CRITICAL
retention 30–730 days
KQL queryable
→ AppTraces table

```
# app_insights.py – Configure OpenTelemetry with Azure Monitor exporter (one file, all three signals)
from azure.monitor.opentelemetry import configure_azure_monitor
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor

# One call sets up traces, metrics, AND logs – all correlated by operationId
configure_azure_monitor(
    connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"],
    cloud_role_name="rag-api", # shows in App Map
)

# Auto-instrument FastAPI (HTTP requests) and httpx (dependency calls to OpenAI)
FastAPIInstrumentor.instrument_app(app)
HTTPXClientInstrumentor().instrument()

# Custom span for RAG query (adds custom attributes visible in App Insights)
```

Fig 13.2 — OpenTelemetry SDK: one configure call instruments traces, metrics, logs

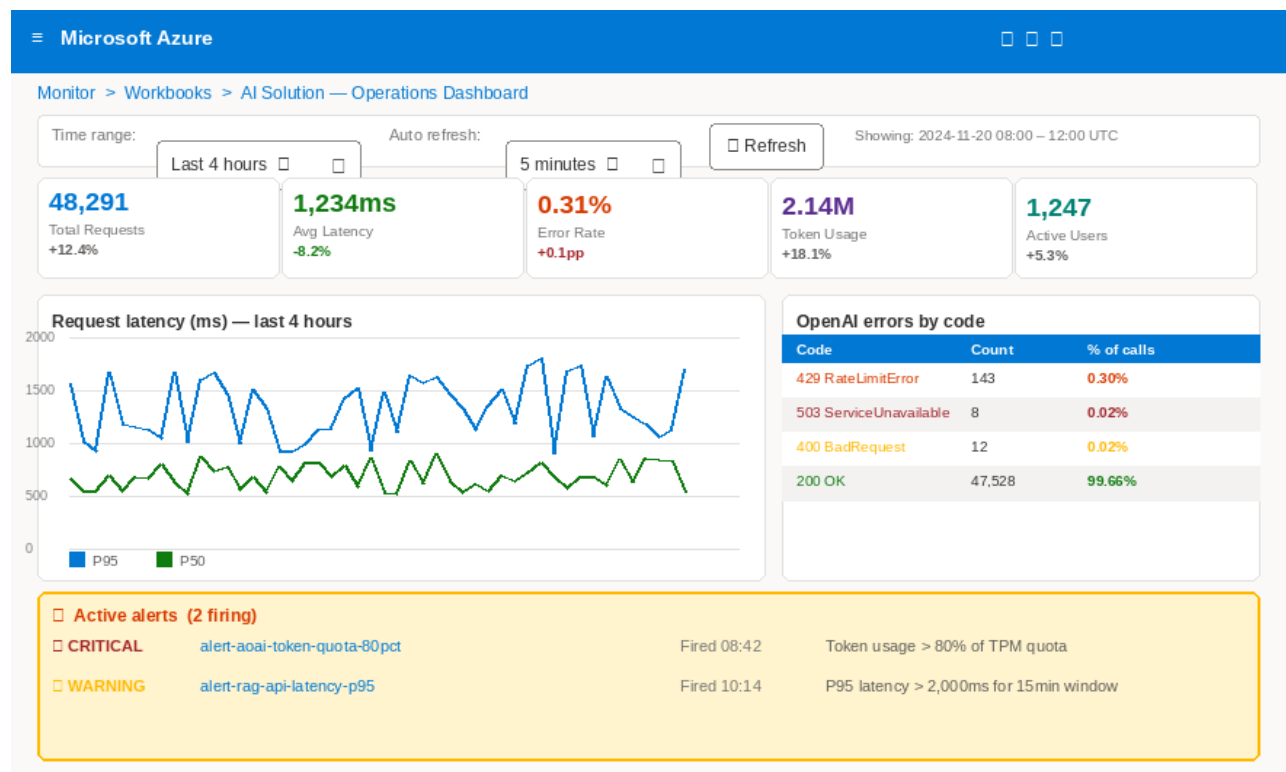


Fig 13.3 — Operations dashboard: KPI cards, latency chart, error log, active alerts

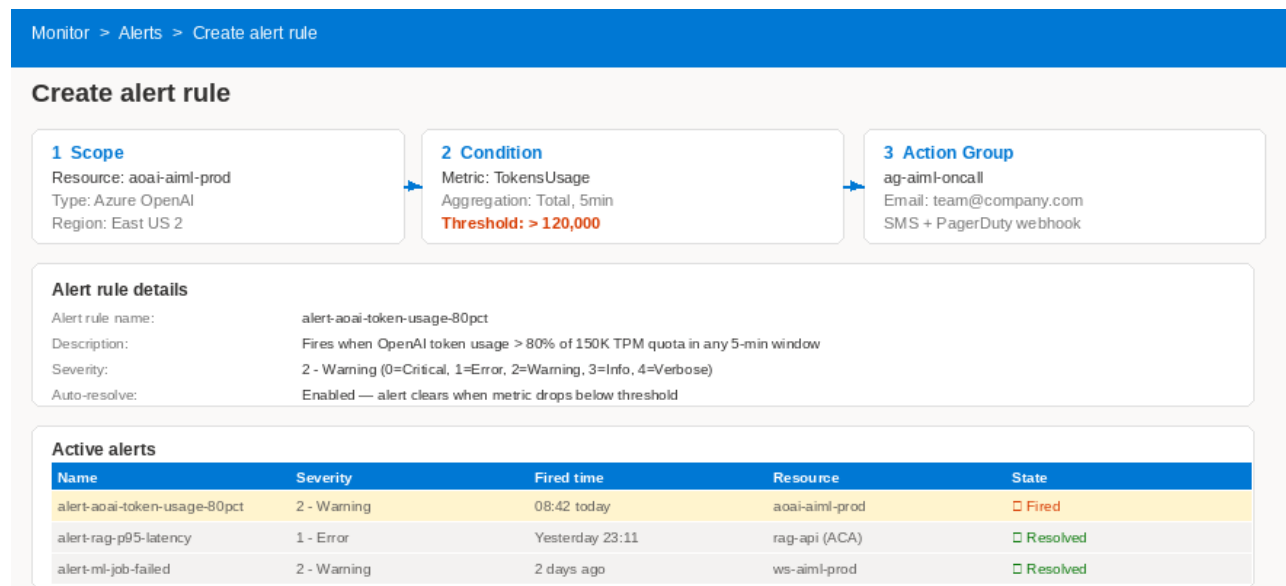


Fig 13.4 — Alert rule: scope, condition (token quota 80%), action group

```
# Setup observability – call once at startup
from azure.monitor.opentelemetry import configure_azure_monitor
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor

configure_azure_monitor(
    connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"],
    cloud_role_name="rag-api",
)
```

```
FastAPIInstrumentor.instrument_app(app)    # auto-traces all HTTP requests
HTTPXClientInstrumentor().instrument()    # auto-traces OpenAI SDK HTTP calls

# Add custom attributes to the auto-created span
@app.post("/api/rag/query")
async def rag_query(body: QueryRequest):
    span = trace.get_current_span()
    span.set_attribute("rag.user_id", body.user_id)
    span.set_attribute("rag.chunks_retrieved", len(chunks))
    span.set_attribute("rag.prompt_tokens", usage.prompt_tokens)

# KQL: latency trend – paste in Monitor → Log Analytics → Logs
// AppDependencies | where TimeGenerated > ago(1h)
// | where Target contains "openai.azure.com"
// | summarize p50=percentile(DurationMs,50), p95=percentile(DurationMs,95)
// by bin(TimeGenerated, 5m) | render timechart
```

Chapter 14: Cost Management

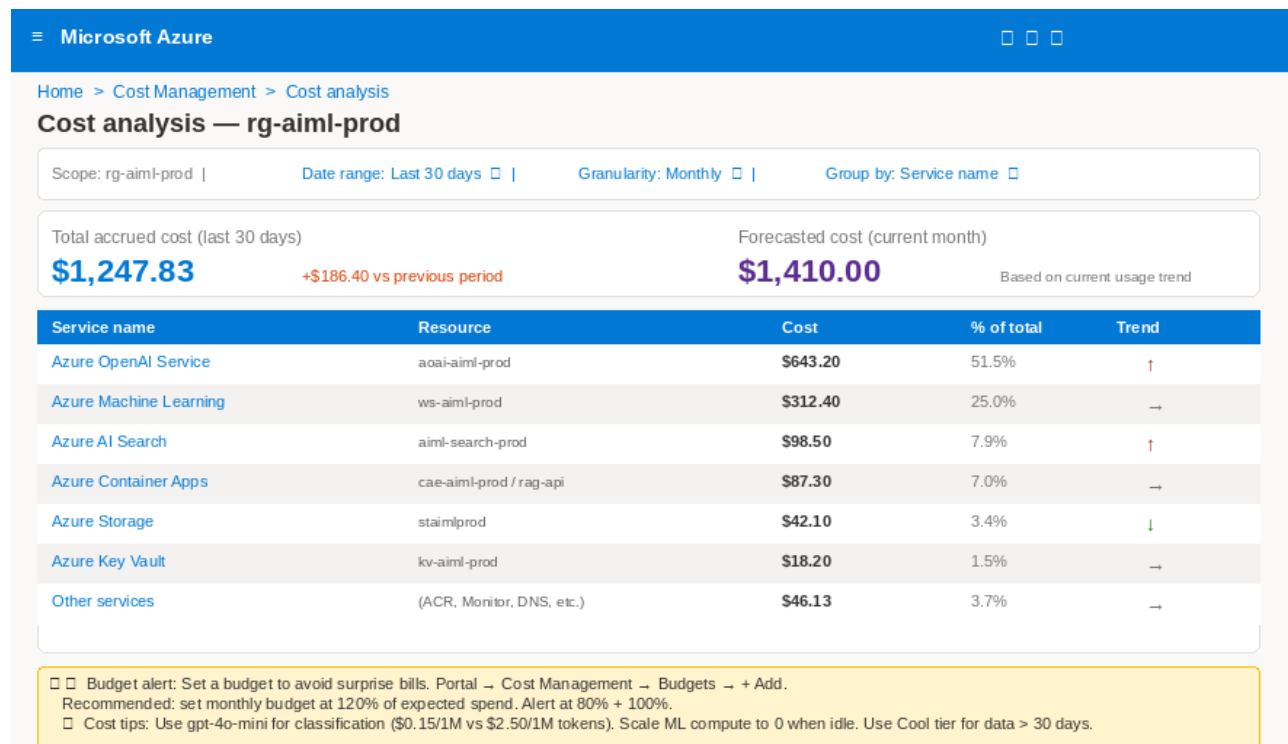


Fig 14.1 — Cost analysis: per-service breakdown with trends and budget alert

Chapter 15: Troubleshooting — Common First-Day Errors

Troubleshooting Guide — Common Azure AI Setup Errors & Fixes

AuthorizationFailed (403)

Cause:
You or your app identity does not have permission to access the resource.

Steps to fix:

- ❑ Check: Azure resource → Access control (IAM) → Role assignments
- ❑ Fix: Add role assignment for your MI/SP: az role assignment create ...
- ❑ Common mistake: Assigned role at subscription scope instead of resource scope
- ❑ Common mistake: Wrong principal ID (object ID vs client ID)
- ❑ Wait 2-5 minutes after role assignment before retrying

SubscriptionNotRegistered (409)

Cause:
The resource provider is not registered for this subscription.

Steps to fix:

- ❑ Portal: Subscriptions → Resource providers → find provider → click Register
- ❑ CLI fix: az provider register --namespace Microsoft.CognitiveServices
- ❑ Then wait ~1 minute and retry your resource creation
- ❑ Check status: az provider show -n Microsoft.CognitiveServices --query registrationState

Cannot connect to endpoint (DNS / network)

Cause:
App can reach public internet but not the Azure private endpoint.

Steps to fix:

- ❑ Test: nslookup aai-ai-ml-prod.openai.azure.com from inside VNet
- ❑ If resolves to 52.x.x.x (public IP) → private DNS zone not linked
- ❑ Fix: Private DNS zone → Virtual network links → add your VNet
- ❑ Fix: Add A record in DNS zone pointing to PE NIC private IP (10.0.4.x)
- ❑ Fix: Check NSG rule allows app subnet → private-endpoints subnet:443

429 RateLimitError (first day)

Cause:
Your deployment TPM quota is too low for your expected load.

Steps to fix:

- ❑ Portal: AI Foundry → Quotas → find your deployment → click Request increase
- ❑ Immediate fix: add exponential backoff retry in your code (tenacity library)
- ❑ Quick workaround: Create a second deployment and load-balance across both
- ❑ Check usage: AI Foundry → Metrics → TokensUsage over last 1 hour

General debug tips: Portal → your resource → Diagnose and solve problems | Resource health | Activity log

Fig 15.1 — Troubleshooting guide: AuthorizationFailed, SubscriptionNotRegistered, DNS, 429

Error	Cause	Fix
403 AuthorizationFailed	RBAC role not assigned, or wrong scope	Resource → Access control (IAM) → Role assignments → verify your MI is listed with correct role at resource scope
409 SubscriptionNotRegistered	Resource provider not registered	Subscriptions → Resource providers → find → Register → wait 1 minute
DNS returns public IP	Private DNS zone not linked to VNet	Private DNS zones → zone → Virtual network links → add your VNet
429 RateLimitError	TPM quota exceeded	AI Foundry → Quotas → deployment → Request increase; add tenacity retry in code
ResourceNotFound 404	Wrong resource name or wrong subscription	az account show to verify subscription; check exact name in portal

💡 When stuck: Portal → your resource → Diagnose and solve problems — runs automated checks. Also check: Activity log (all recent changes) and Resource health (Azure service-side outages).

PART 5: AI USE CASES — End-to-End Examples

Chapter 16: RAG Chatbot — Enterprise Knowledge Q&A

Enterprise RAG Architecture on Azure

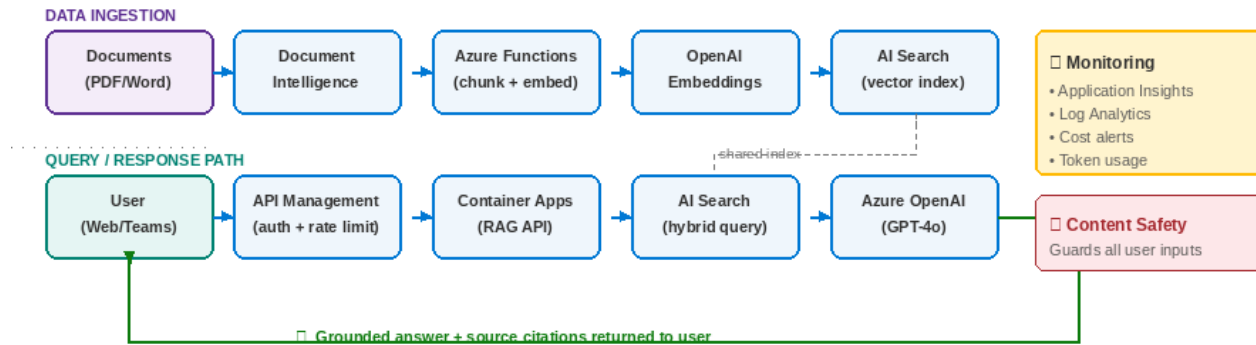


Fig 16.1 — RAG architecture: ingestion path (top) and query/response path (bottom)

RAG Document Ingestion Pipeline — From File Upload to Searchable Index



Fig 16.2 — RAG ingestion workflow: 7 steps from upload to searchable index

```

async def ingest(blob_url: str, tenant_id: str) -> None:
    """Blob created → extract → chunk → embed → index"""
    text = extract_with_doc_intel(blob_url)
    chunks = [" ".join(text.split()[i:i+400]) for i in range(0, len(text.split()), 340)]
    embeddings = aoai.embeddings.create(model="text-embed-large", input=chunks)
    search.merge_or_upload_documents([
        {"id": f"{blob_url.split('/')[0]}_{i}",
         "content": c, "embedding": e.embedding,
         "source_url": blob_url, "tenant_id": tenant_id}
        for i,(c,e) in enumerate(zip(chunks, embeddings.data))
    ])

async def rag_query(question: str, tenant_id: str) -> dict:
    """Embed → retrieve → generate → store"""
    q_emb = aoai.embeddings.create(model="text-embed-large",
                                    input=[question]).data[0].embedding

    chunks = list(search.search(
        search_text=question,

```

```
vector_queries=[VectorizedQuery(vector=q_emb, k_nearest_neighbors=5,
fields="embedding")],
filter=f"tenant_id eq '{tenant_id}'", # CRITICAL: data isolation
query_type="semantic", semantic_configuration_name="default", top=5))

ctx = "\n\n".join(f"[{i+1}] {r['content']}" for i,r in enumerate(chunks))

resp = aoai.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role":"system","content":f"Answer ONLY from context. Cite [1],[2].\n\n{ctx}"},
        {"role":"user","content":question}],
    max_tokens=1500, temperature=0.1)

return {"answer": resp.choices[0].message.content,
        "sources": [r["source_url"] for r in chunks]}
```


Chapter 17: Document Intelligence Pipeline

Use Case 2: Document Intelligence Pipeline — PDF → Extract → Classify → Store

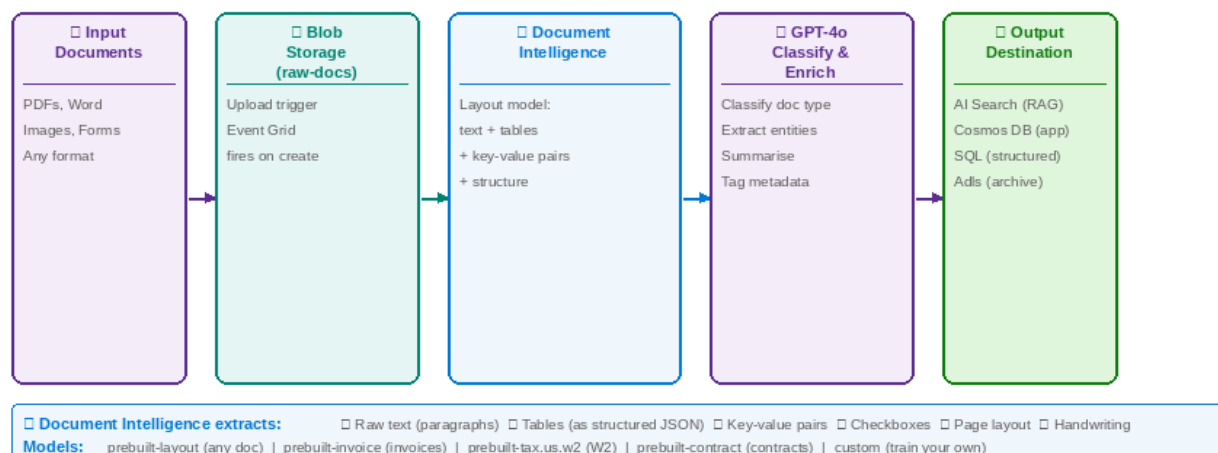


Fig 17.1 — Document pipeline: upload → extract → classify → route to system

```
from azure.ai.documentintelligence import DocumentIntelligenceClient
from pydantic import BaseModel

doc_intel = DocumentIntelligenceClient(os.environ["DOC_INTEL_ENDPOINT"],
DefaultAzureCredential())

class DocAnalysis(BaseModel):
    doc_type: str          # invoice, contract, receipt, form, other
    vendor: str | None; total: float | None
    key_entities: list[str]; confidence: float

async def process_document(blob_url: str) -> dict:
    # 1. Extract with Document Intelligence
    result = doc_intel.begin_analyze_document_from_url("prebuilt-layout", blob_url).result()
    text = " ".join(p.content for p in (result.paragraphs or []))
    kvps = {kv.key.content: kv.value.content for kv in (result.key_value_pairs or []) if
kv.value}

    # 2. Classify with GPT-4o-mini (cheaper for classification)
    resp = aoai.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": f"Classify:\n{text[:3000]}\nKVPs:{kvps}"},
        response_format=DocAnalysis)
    analysis = resp.choices[0].message.parsed

    # 3. Route based on classification
    if analysis.doc_type == "invoice" and analysis.confidence > 0.85:
        await store_in_accounts_payable(analysis, blob_url)
    else:
        await queue_for_human_review(blob_url, analysis)
    return analysis.model_dump()
```

Chapter 18: AI Customer Support

Use Case 3: AI-Powered Customer Support — Classify → Suggest → Escalate

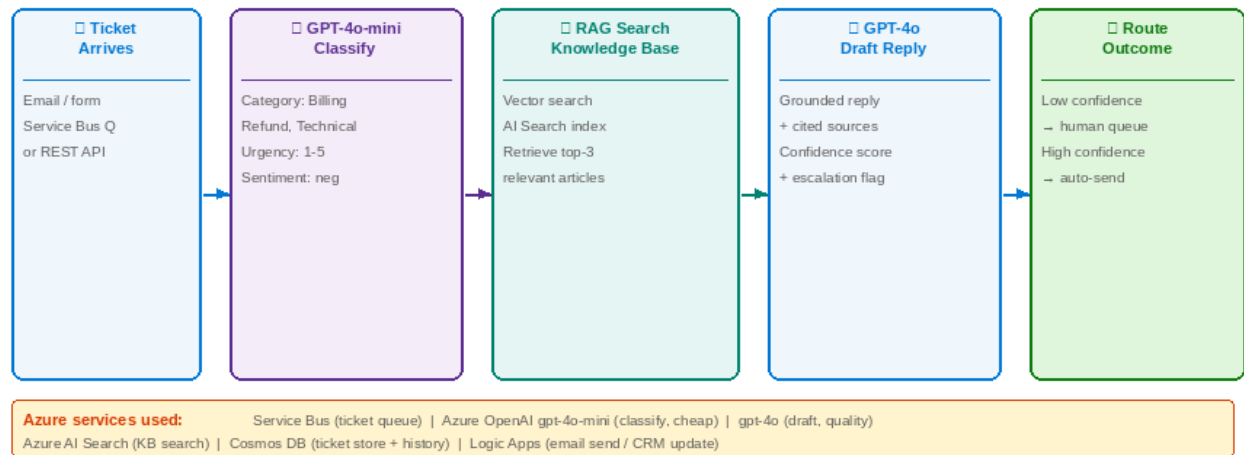


Fig 18.1 — Classify ticket → search KB → draft reply → auto-send or human queue

Chapter 19: Semantic Search

Use Case 4: Semantic Product Search — Replace keyword with meaning-aware search

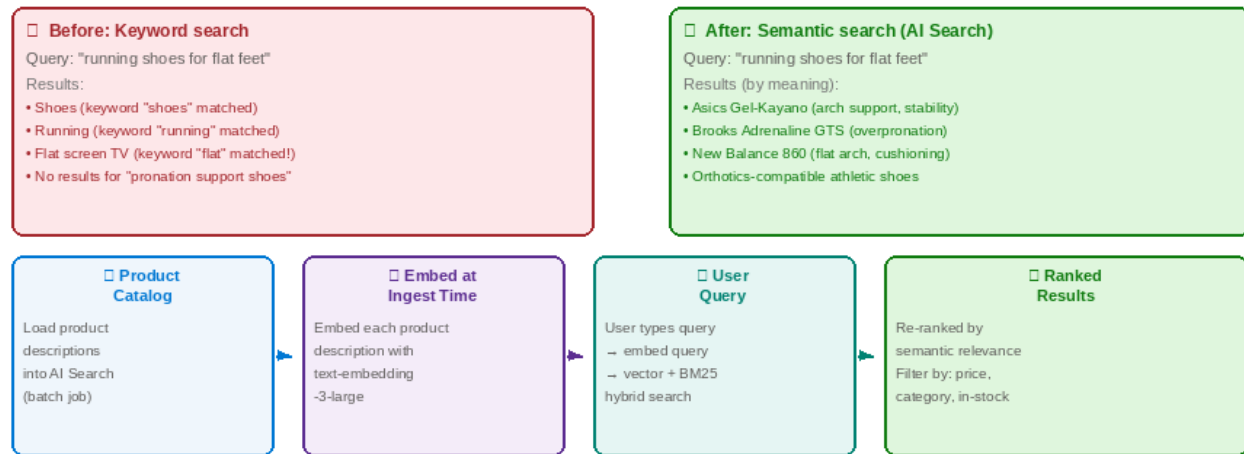


Fig 19.1 — Semantic search: before/after comparison, product embedding + query flow

Chapter 20: Batch Content Generation

```
# OpenAI Batch API — 50% cost saving for async workloads
import json, time

def batch_generate(items: list[dict]) -> dict:
    requests = [{"custom_id": f"item-{i}", "method": "POST",
                  "url": "/v1/chat/completions",
                  "body": {"model": "gpt-4o-mini",
                           "messages": [{"role": "user", "content": item["prompt"]}],
                           "max_tokens": 200}}
                 for i, item in enumerate(items)]

    jsonl = "\n".join(json.dumps(r) for r in requests)
    f = aoai.files.create(file=("batch.jsonl", jsonl.encode()), purpose="batch")

    batch = aoai.batches.create(
        input_file_id=f.id, endpoint="/v1/chat/completions", completion_window="24h")

    while batch.status not in ["completed", "failed", "cancelled"]:
        time.sleep(300) # poll every 5 minutes
        batch = aoai.batches.retrieve(batch.id)

    if batch.output_file_id:
        results = {}
        for line in aoai.files.content(batch.output_file_id).text.split("\n"):
            if line:
                r = json.loads(line)
                results[r["custom_id"]] = r["response"]["body"]["choices"][0]["message"]
        ["content"]
    return results
```

APPENDICES

Appendix A: CLI Quick Reference

Task	Command
Login	az login && az account show --query "{sub:id, tenant:tenantId}" -o table
Create RG	az group create -n rg-aiml-prod -l eastus2 --tags env=prod team=ai-ml
Register providers	for P in Microsoft.CognitiveServices Microsoft.Search Microsoft.App; do az provider register -n \$P; done
Create OpenAI	az cognitiveservices account create -n aoai-aiml-prod -g rg-aiml-prod --kind OpenAI --sku S0 -l eastus2 --custom-domain aoai-aiml-prod
Create AI Search	az search service create -n aiml-search-prod -g rg-aiml-prod --sku standard
Create Key Vault	az keyvault create -n kv-aiml-prod -g rg-aiml-prod --enable-rbac-authorization --enable-purge-protection
Create ACR	az acr create -n acraimlprod -g rg-aiml-prod --sku Premium --admin-enabled false
Build + push image	az acr build --registry acraimlprod --image rag-api:\${git rev-parse --short HEAD} .
Create Container App	az containerapp create -n rag-api -g rg-aiml-prod --environment cae-aiml-prod --image acraimlprod.azurecr.io/rag-api:latest --target-port 8000 --ingress external --system-assigned
Assign RBAC role	az role assignment create --role "Cognitive Services OpenAI User" --assignee <principal-id> --scope <resource-id>
List role assignments	az role assignment list --assignee <principal-id> --all -o table
Test PE DNS	nslookup aoai-aiml-prod.openai.azure.com (should return 10.0.4.x)

Appendix B: Python SDK Quick Reference

```
# All imports for an Azure AI Python project
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.keyvault.secrets import SecretClient
from azure.storage.blob import BlobServiceClient
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.monitor.opentelemetry import configure_azure_monitor
import os

# One credential – auto-detects MI (Azure), az login (local), env vars (CI/CD)
cred = DefaultAzureCredential()

# Azure OpenAI – keyless auth (NEVER use openai.api_key in production)
aoai = AzureOpenAI(
    azure_endpoint=os.environ["OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        cred, "https://cognitiveservices.azure.com/.default"),
    api_version="2024-08-01-preview")

# AI Search
search = SearchClient(os.environ["SEARCH_ENDPOINT"], "rag-docs-index", cred)

# Key Vault
kv = SecretClient("https://kv-aiml-prod.vault.azure.net/", cred)

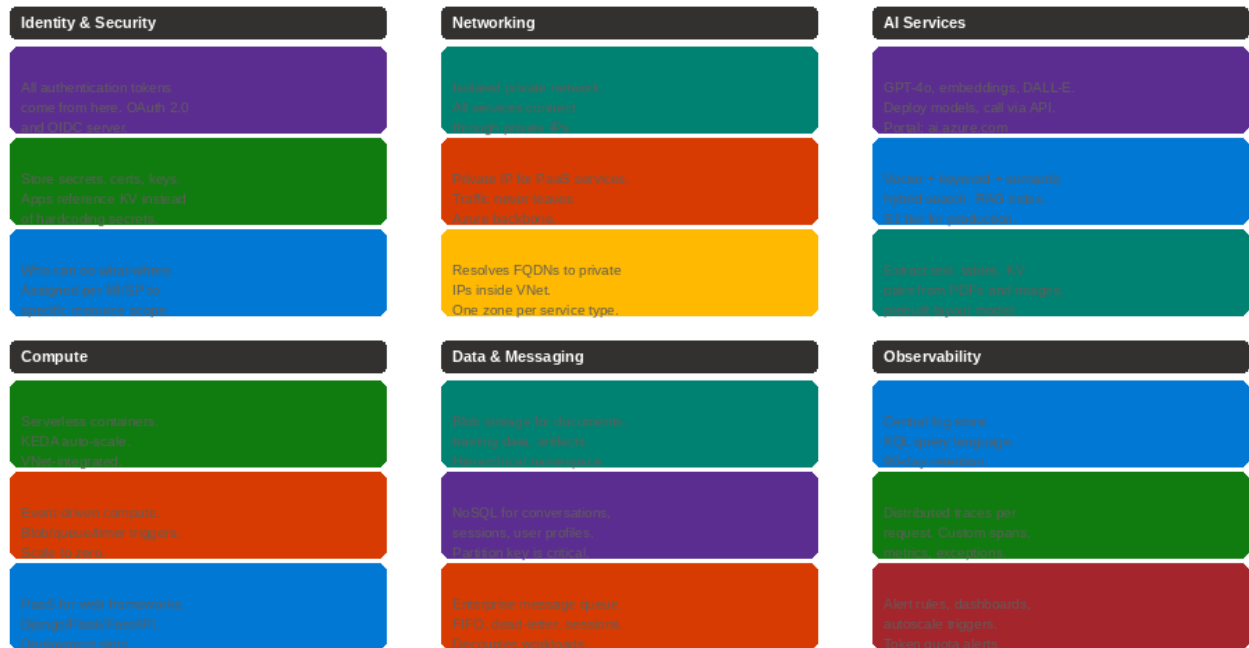
# Blob Storage
blob = BlobServiceClient("https://staimlprod.blob.core.windows.net", cred)

# Document Intelligence
doc = DocumentIntelligenceClient(os.environ["DOC_INTEL_ENDPOINT"], cred)

# Observability (call once at startup)
configure_azure_monitor(connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"])
```

Appendix C: Service Relationships Map

Azure AI/ML Services — What Each Does and How They Connect



All service-to-service calls use Managed Identity → RBAC role → Entra ID token → Bearer auth. NO hardcoded secrets anywhere.

Fig C.1 — How all Azure AI services relate: identity, networking, compute, data, observability

How Azure AI Services Relate — Dependencies & Integration Map

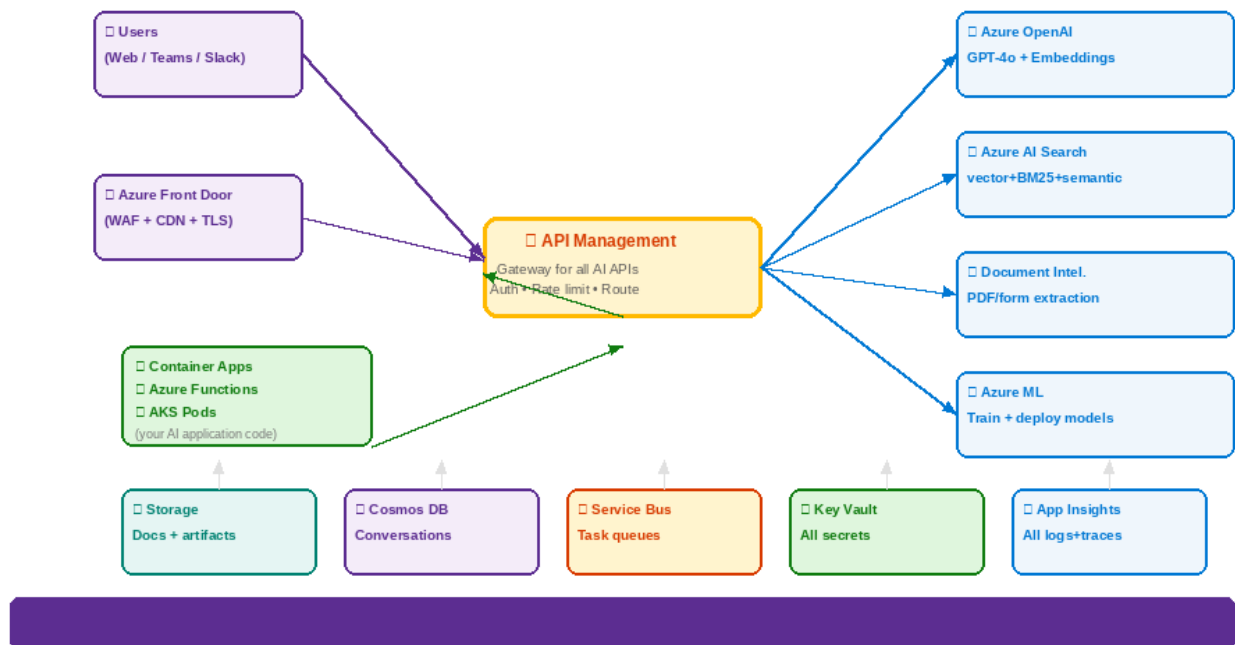


Fig C.2 — Service integration map showing dependencies and connection patterns

— End of Handbook (Seventh Edition) —